

The Machine

By

Matt Reid

June 2017

Introduction and Thanks

The following paper is essentially a more detailed retelling of the previous paper entitled 'Valkyrie' that has been languishing in the file uploads area of the Yahoo Kryptos Group since October 2013. The original paper wasn't widely publicised as it was a place holder, summarizing discoveries to that date that fell well short of any meaningful conclusions about the possible final methods of decryption. That paper also omitted any mention of the holistic nature of the sculpture itself. Since that time we have had another clue revealed by Jim Sanborn by way of the word CLOCK for positions 70 through to 74. Serendipity has also played a part in that intervening period in having discovered Carl Sagan's novel 'Contact' over the 2013 Christmas holiday season. I have amended the original paper to show how the book is an essential resource that directs us to lines of enquiry beyond the clues given in parts K1-K3. This paper has been named as 'The Machine' in order to pay homage to the concept in Sagan's Contact novel that once 'the message' was decoded, that it laid bare instructions for how to build a machine to reconnect mankind to the cosmos.

As usual the readership is aimed primarily at those in the Kryptos K4 community who are already conversant with the cipher and plain texts of sections K1-K3 as well as the associated tools and techniques for the relevant decryptions. This is a lengthy paper, but no apologies are given for that other than for it being perhaps too brief in places, but I have endeavoured to include enough detail to make the reconstruction of the results as clear as possible. There will no doubt be some mistakes and some areas where I could have expanded more with clearer examples and diagrams, but the general premise presented herein is sound and self-consistent, so please feel free to copy any material in this paper in order to illustrate any corrections as part of your own work. I'm happy for this work to be redistributed in its entirety or drawn upon in parts, but please include a citation. I have not submitted this paper to Jim Sanborn for verification as nature ultimately completes that particular job for us.

As with sections K1, K2 and K3 anyone wishing to repeat these results for themselves needs only the decryption tableau, a pen and some paper (although a computer and relevant software is recommended). Source code is provided (with no copyright or warranty, so use at your own risk) in the appendix for people looking to reconstruct 'The Machine' and play around with different decryption algorithms. The K3 decryption algorithm is included out of the box as part of this source code, but as the eclipse of 21st August 2017 looms before us I'm hopeful this paper may inspire many newcomers to take up the challenge to find a way to use Kryptos to predict this event as an exposition of its underlying function. The rewards of such an endeavour may be to extend the findings of this paper even further, but for my part I'm satisfied that the results contained herein will at last break the long hiatus since K3 was publicly decoded.

This introduction would be incomplete without a sincere expression of gratitude to Jim Sanborn, Ed Scheidt, their patrons and any other unnamed parties behind the scenes for seeding the world with such a constructive and educational curiosity.

Lastly I'd like to thank Elonka Dunin who took the time out to cast an eye over my initial paper back in 2010. Anyone unfamiliar with Kryptos that wants additional background may also want to visit her Kryptos site at <http://www.elonka.com/kryptos> before proceeding any further with this document.

Abstract

A number of what are thought to be novel discoveries are hereby put into the public domain for wider discussion and verification. The core finding is that the Kryptos cipher text is layered on top of a numerical surface that is independent of the lettering itself. This surface represents a hardware model on which to perform an analogue computation of the moons cycles in relation to the solar day and year. As a result of this it is conjectured that the sculpture is in fact a symbolic representation of the Antikythera mechanism. It will be shown that the sculpture is capable of recording time at the level of seconds, minutes, hours, days and weeks. It can further be used to capture the calendar, sidereal and synodic months. The largest time span it records is the 76 year Callippic cycle, which is composed of 4 Metonic cycles each of 19 years in length. The ability to record these cycles would also imply that it has the ability to accurately predict eclipse cycles. Strong indications are also given that additional problems will need to be solved in order to fully decrypt K4/K5 that relate to data manipulation and the discovery of the correct algorithmic process for letter selection. Though there are many new insights in this paper the most important is that of putting a new perspective on the sculpture in that it is far more than a redundant copper scroll once the K4/K5 plaintext is known and reveals instead a new function that takes it into perpetuity.

Additional Background Materials

For anyone unfamiliar with the novel 'Contact' by Carl Sagan, I would suggest picking up a copy, not only for the fact that the story stands up by itself as a fantastic read in its own right, but also for the reason that it will resonate strongly with anyone immersed in this particular adventure. The book is still widely available and can be searched worldwide by its ISBN number which is: 0-09-946950-2. Please note that the plot of the Jodie Foster film differs significantly at the end of the story, so please take the time out to read the book in its entirety (you won't be disappointed!).

To summarize the story very briefly for the purposes of this paper, the story revolves around the personnel involved in a SETI (Search for Extra Terrestrial Intelligence) programme that are using radio telescopes to scan deep space. The group eventually find a signal that can only be from an extra-terrestrial source. The 'Message' as it comes to be known is composed of repeating prime numbers, but further information is found to be encoded through the use of polarisation and phase modulation. How this is relevant to K4 will become apparent in the following paper. Most importantly the story relates how the 'Message' once decrypted leads to plans for the construction of a 'Machine'.

An additional resource that is essential to understanding the hardware model in use is a good reference guide to the mechanics of the Antikythera mechanism. An excellent book on the subject 'A Portable Cosmos', has recently been released by Alexander Jones ISBN 978-0-19-973934-9 and contains many new discoveries and insights many of which were not known or verified in 1990.

Part I

Background

The following information on the background of the Kryptos sculpture was gathered from a range of internet sources.

Kryptos was dedicated on **November the 3rd 1990** on the grounds of the CIA headquarters in Langley Virginia. The main body of the sculpture is a worked copper scroll by the Artist Jim Sanborn who was commissioned by the CIA to create a series of installation pieces for the buildings courtyard. He was additionally loaned the services of a retiring CIA cryptographic expert by the name of Ed Scheidt who was on hand to provide assistance with the cryptographic elements of the underlying text.

Since 1990 there have been three public acknowledgments in relation to the decryption of parts K1, K2 and K3.

A civilian computer scientist named Jim Gillogly went public with a decryption of parts K1, K2 and K3 in 1999 having used computational analysis to arrive at the plain texts.

After this announcement it was revealed that a CIA employee named David Stein was recognised within the CIA as having cracked parts K1, K2 and K3 using a pencil and paper technique in 1998.

It was later announced in 2005 that an NSA team led by Ken Miller, Dennis McDaniels and two other unnamed individuals had deciphered parts K1, K2 & K3 in late 1992 using computational analysis.

In the years since Gillogly's announcement a mystique has continued to grow around the unsolved K4 cipher text. This is partly due to its resistance to yield its final secrets, but also the mass of interest aroused worldwide by Dan Brown's code sleuthing novels and the growing pervasiveness of cryptography in modern life. As a result there has been a mass of internet based collaboration in the public domain ever since, whereby common interest groups and individuals have persistently poured over the clues left in the plain texts of parts K1, K2 and K3 in search of possible breaks.

For anyone not familiar with the layout of the sculptures original cipher text or the plain texts of K1, K2 and K3 then they read as follows.

The Sculpture Text

The text as it appears on the sculpture has been lifted wholesale from Elonka's website (www.elonka.com/kryptos) and is presented in full below.

Complete Cipher Text and Decryption Tableau as it appears on the sculpture.

EMUFPHZLRFAXYUSDJKZLDKRNHGNFIVJ
YQTQUXQBQVYUUVLLTREVJYQTMKYRDMFD
VFPJUDEEHZWETZYVGWHKKQETGFQJNCE
GGWHKK?DQMC PFQZDQMMIAGPFXHQRLG
TIMVMZJANQLVKQEDAGDVFRPJUNGEUNA
QZGZLECGYUXUEENJTBILBQCRTBJDFHRR
YIZETKZEMVDUFKSJHKFWHKUWQLSZFTI
HHDDDUVH?DWKBFUFPWNTDFIYCUQZERE
EVLDKFEZMOQQJLTUGSYQPFEUNLAVIDX
FLGGTEZ?FKZBSFDQVGOGIPUFXHHDRKF
FHQNTGPUAECNUVPDJMQCLQUMUNEDFQ
ELZZVRRGKFFVOEEXBDMVPNFQXEZLGRE
DNQFMPNZGLFLPMRJQYALMGNUVPDXVKP
DQUMEBEDMHDAFMJGZNUPLGEWJLLAETG

ABCDEFGHIJKLMNOPQRSTUVWXYZABCD
AKRYPTOSABCDEFGHIJLMNQUVWXXZKRY
BRYPTOSABCDEFGHIJLMNQUVWXXZKRYPT
CYPTOSABCDEFGHIJLMNQUVWXXZKRYPTO
DPTOSABCDEFGHIJLMNQUVWXXZKRYPTOS
ETOSABCDEFGHIJLMNQUVWXXZKRYPTOSA
FOSABCDEFGHIJLMNQUVWXXZKRYPTOSAB
GSABCDEFGHIJLMNQUVWXXZKRYPTOSABC
HABCDEFGHIJLMNQUVWXXZKRYPTOSABCD
IBABCDEFGHIJLMNQUVWXXZKRYPTOSABCDE
JABCDEFGHIJLMNQUVWXXZKRYPTOSABCDEF
KABCDEFGHIJLMNQUVWXXZKRYPTOSABCDEF
LEFGHIJLMNQUVWXXZKRYPTOSABCDEFGHI
MFGHIJLMNQUVWXXZKRYPTOSABCDEFGHI

ENDYAHROHNLSRHEOCPTEOIBIDYSHNAIA
CHTNREYULDSLLSLLNOHSNOSMRWXMNE
TPRNGATIHNRRAPESLNNELEBLPIIACAE
WMTWNDITEENRAHCTENEUDRETNHAEOE
TFOLSEDTIWENHAEIOYTEYQHEENCTAYCR
EIFTBRSPAMHHEWENATAMATEGYEERLB
TEEFOASFIOTUETUAEOTOARMAEERTNRTI
BSEDDNIAAHTTMSTEWPIEROAGRIEFEB
AECTDDHILCEIHSITEGOEAOSDDRYDLORIT
RKLMLHAGTDHARDPNEOHMGFMFEUHE
ECDMRIPFEIMEHNLSSTTRTVDOHW?OBKR
UOXOGHULBSOLIFBBWFLRVQPRNGKSSO
TWTQSJSSEKZZWATJKLUDIAWINFBNYP
VTTMZFPKWGDKZXTJCDIGKUHUAUEKCAR

NGHIJLMNQUVWXXZKRYPTOSABCDEFGHIJL
OHIJLMNQUVWXXZKRYPTOSABCDEFGHIJL
PIJLMNQUVWXXZKRYPTOSABCDEFGHIJLM
QJLMNQUVWXXZKRYPTOSABCDEFGHIJLMN
RLMNQUVWXXZKRYPTOSABCDEFGHIJLMNQ
SMNQUVWXXZKRYPTOSABCDEFGHIJLMNQU
TNQUVWXXZKRYPTOSABCDEFGHIJLMNQUV
UQUVWXXZKRYPTOSABCDEFGHIJLMNQUVW
VUVWXXZKRYPTOSABCDEFGHIJLMNQUVWX
VWVWXXZKRYPTOSABCDEFGHIJLMNQUVWX
XWVWXXZKRYPTOSABCDEFGHIJLMNQUVWX
YXVWXXZKRYPTOSABCDEFGHIJLMNQUVWX
ZZKRYPTOSABCDEFGHIJLMNQUVWXXZKRY
ABCDEFGHIJKLMNOPQRSTUVWXYZABCD

K1 is highlighted in Mauve, K2 in Blue, K3 in Green and K4 in Red

Known Plain Text for Sections K1, K2 and K3

K 1 (this was encrypted using the Keyword PALIMPSEST)

BETWEEN SUBTLE SHADING AND THE ABSENCE OF LIGHT LIES THE NUANCE OF IQLUSION

K2 (this was encrypted using the Keyword ABSCISSA) Note: This was publicly corrected by Jim Sanborn on April 19th, 2006 who intervened to point out the IDBYROWS should read XLAYERTWO

IT WAS TOTALLY INVISIBLE HOWS THAT POSSIBLE? THEY USED THE EARTHS MAGNETIC FIELD X THE INFORMATION WAS GATHERED AND TRANSMITTED UNDERGUUND TO AN UNKNOWN LOCATION X DOES LANGLEY KNOW ABOUT THIS? THEY SHOULD ITS BURIED OUT THERE SOMEWHERE X WHO KNOWS THE EXACT LOCATION? ONLY WW THIS WAS HIS LAST MESSAGE X THIRTY EIGHT DEGREES FIFTY SEVEN MINUTES SIX POINT FIVE SECONDS NORTH SEVENTY SEVEN DEGREES EIGHT MINUTES FORTY FOUR SECONDS WEST X LAYER TWO

K 3 (this section was not encrypted using the Kryptos Tableau, but instead made use of a 192 character skip sequence to encrypt the plain text)

SLOWLY DESPARATLY SLOWLY THE REMAINS OF PASSAGE DEBRIS THAT ENCUMBERED THE LOWER PART OF THE DOORWAY WAS REMOVED WITH TREMBLING HANDS I MADE A TINY BREACH IN THE UPPER LEFT HAND CORNER AND THEN WIDENING THE HOLE A LITTLE I INSERTED THE CANDLE AND PEERED IN THE HOT AIR ESCAPING FROM THE CHAMBER CAUSED THE FLAME TO FLICKER BUT PRESENTLY DETAILS OF THE ROOM WITHIN EMERGED FROM THE MIST X CAN YOU SEE ANYTHING Q

In addition to the sculpture there are also installations around the courtyard with embedded morse code fragments. Once interpreted these read.

SOS, LUCID MEMORY, T IS YOUR POSITION, SHADOW FORCES, VIRTUALLY INVISIBLE, DIGETAL INTERPRETATU, and RQ (which maybe alternatively be YR depending on which way up its read).

Interventions post decryption of K1-K3

Issues with the decryption of K2

With regards the correct decryption of K2, Jim Sanborn the artist that created Kryptos intervened personally on April 19th 2006, contacting the group moderators in order to say, "No, that last part is wrong." Elonka's Kryptos website at www.elonka.com/kryptos relays the following about this incident.

"He also indicated that there was a missing character on the sculpture, probably something that would have resulted in a plaintext "X" before that section. He said that he had thought that with the missing character, the section in question would have come out to be an unintelligible scramble. Instead, he was astounded to see that by sheer chance, the resulting random text had turned out to be apparently intelligible English, "ID BY ROWS", although that was not what was intended."

The Berlin Clue

On the 21st November 2010 Jim Sanborn once again intervened, but in this instance it was to reveal a clue. Sanborn indicated that a six letter string between positions 64 through to 69 of the Kryptos plaintext for K4 would read BERLIN once decrypted.

The Clock Clue

On the 20th November 2014, Jim Sanborn revealed another clue in the form of the word CLOCK being the plaintext for positions 70 through to 74. The fact that 11 letters of the plaintext were now known now massively reduced the odds of being able to find suitable decryption schemes.

Part II

It will become evident throughout each step that the whole of the Kryptos sculpture represents a larger body of work than previously realised and is only eventually seen for what it is by solving multiple smaller puzzles. Some of these steps are more significant than others, but in total they add up to a single conclusion that is both irrefutable and intellectually satisfying. Viewed as a whole it will be seen that the piece quite literally steps us through the art and history of cryptography itself. This is done quite brilliantly and bears testament to the intellect and artistry of both Scheidt and Sanborn. I suspect that a further study of the cipher and plain texts after this paper has been distributed will yield many more encryption methods, some of which may light a way to a decryption path that does not require foreknowledge of the BERLIN and CLOCK clues respectively.

Observation of symmetry and spaces

The cipher text is initially laid out in the following form in order to highlight the symmetry relationship between K1 and K2 in the top 14 rows compared against K3 and K4 in the bottom 14. Observe also the spaces on the right hand side. Note the positioning of the seemingly extraneous T that sits outside of the grid system in row 23.

E	M	U	F	P	H	Z	L	R	F	A	X	Y	U	S	D	J	K	Z	L	D	K	R	N	S	H	G	N	F	I	V	J	
Y	Q	T	Q	U	X	Q	B	Q	V	Y	U	V	L	L	T	R	E	V	J	Y	Q	T	M	K	Y	R	D	M	F	D		
V	F	P	J	U	D	E	E	H	Z	W	E	T	Z	Y	V	G	W	H	K	K	Q	E	T	G	F	Q	J	N	C	E		
G	G	W	H	K	K	?	D	Q	M	C	P	F	Q	Z	D	Q	M	M	I	A	G	P	F	X	H	Q	R	L	G			
T	I	M	V	M	Z	J	A	N	Q	L	V	K	Q	E	D	A	G	D	V	F	R	P	J	U	N	G	E	U	N	A		
Q	Z	G	Z	L	E	C	G	Y	U	X	U	E	E	N	J	T	B	J	L	B	Q	C	R	T	B	J	D	F	H	R	R	
Y	I	Z	E	T	K	Z	E	M	V	D	U	F	K	S	J	H	K	F	W	H	K	U	W	Q	L	S	Z	F	T	I		
H	H	D	D	D	U	V	H	?	D	W	K	B	F	U	F	P	W	N	T	D	F	I	Y	C	U	Q	Z	E	R	E		
E	V	L	D	K	F	E	Z	M	O	Q	Q	J	L	T	T	U	G	S	Y	Q	P	F	E	U	N	L	A	V	I	D	X	
F	L	G	G	T	E	Z	?	F	K	Z	B	S	F	D	Q	V	G	O	G	I	P	U	F	X	H	H	D	R	K	F		
F	H	Q	N	T	G	P	U	A	E	C	N	U	V	P	D	J	M	Q	C	L	Q	U	M	U	N	E	D	F	Q			
E	L	Z	Z	V	R	R	G	K	F	F	V	O	E	E	X	B	D	M	V	P	N	F	Q	X	E	Z	L	G	R	E		
D	N	Q	F	M	P	N	Z	G	L	F	L	P	M	R	J	Q	Y	A	L	M	G	N	U	V	P	D	X	V	K	P		
D	Q	U	M	E	B	E	D	M	H	D	A	F	M	J	G	Z	N	U	P	L	G	E	W	J	L	L	A	E	T	G		
E	N	D	Y	A	H	R	O	H	N	L	S	R	H	E	O	C	P	T	E	O	I	B	I	D	Y	S	H	N	A	I	A	
C	H	T	N	R	E	Y	U	L	D	S	L	L	S	L	L	N	O	H	S	N	O	S	M	R	W	X	M	N	E			
T	P	R	N	G	A	T	I	H	N	R	A	R	P	E	S	L	N	N	E	L	E	B	L	P	I	I	A	C	A	E		
W	M	T	W	N	D	I	T	E	E	N	R	A	H	C	T	E	N	E	U	D	R	E	T	N	H	A	E	O	E			
T	F	O	L	S	E	D	T	I	W	E	N	H	A	E	I	O	Y	T	E	Y	Q	H	E	E	N	C	T	A	Y	C	R	
E	I	F	T	B	R	S	P	A	M	H	H	E	W	E	N	A	T	A	M	A	T	E	G	Y	E	E	R	L	B			
T	E	E	F	O	A	S	F	I	O	T	U	E	T	U	A	E	O	T	O	A	R	M	A	E	E	R	T	N	R	T	I	
B	S	E	D	D	N	I	A	A	H	T	T	M	S	T	E	W	P	I	E	R	O	A	G	R	I	E	W	F	E	B		
A	E	C	T	D	D	H	I	L	C	E	I	H	S	I	T	E	G	O	E	A	O	S	D	D	R	Y	D	L	O	R	I	T
R	K	L	M	L	E	H	A	G	T	D	H	A	R	D	P	N	E	O	H	M	G	F	M	F	E	U	H	E				
E	C	D	M	R	I	P	F	E	I	M	E	H	N	L	S	S	T	T	R	T	V	D	O	H	W	?	O	B	K	R		
U	O	X	O	G	H	U	L	B	S	O	L	I	F	B	B	W	F	L	R	V	Q	Q	P	R	N	G	K	S	S	O		
T	W	T	Q	S	J	Q	S	S	E	K	Z	Z	W	A	T	J	K	L	U	D	I	A	W	I	N	F	B	N	Y	P		
V	T	T	M	Z	F	P	K	W	G	D	K	Z	X	T	J	C	D	I	G	K	U	H	U	A	U	E	K	C	A	R		

Examination of the plaintext for hints

Like all encryption schemes, K1, K2 and K3 have weaknesses that made them vulnerable to cryptographic analysis and attack. K4 has so far resisted any form of conventional cryptographic attack, which implies the sculpture text taken at face value is either not a computable quantity or if it is then uses an encryption scheme that leaves no statistical artefacts in its wake that can be exploited by the cryptographer in order to reconstruct the underlying scheme.

The alternative is that we have to manipulate the sculpture text in some way in order to make it a computable surface and that K4 somehow interacts with this, rather than being a distinct cipher in its own right.

The necessary manipulations that we need to make in order to make the sculpture computable are given by a series of clues present in the plain text of K1, K2 and K3.

There are multiple 'lucky breaks' that can be made by analysing different clues in the plaintext, but significantly any of these initial entry points lead to the same eventual conclusion. To illustrate this point I'll start with a three part clue in K3 that is initially cryptic, but can then be asserted mathematically to arrive at our first meaningful break.

I MADE A **TINY** BREACH IN **THE UPPER LEFT HAND CORNER** AND THEN WIDENING THE HOLE A LITTLE I INSERTED THE CANDLE AND PEERED IN THE **HOT AIR ESCAPING** FROM THE CHAMBER

We can approach two aspects of this clue cryptically and one at face value. The **HOT AIR ESCAPING** can be thought of as the FUME's leaving the previously sealed chamber. As we're looking at the puzzle through this same cryptic lens we can also assert that the **TINY** breach might alternatively be a synonym of **TINY** in the form of the word **PUNY**. Lastly the location of this breach is mentioned as being in the top left hand corner. Given that the keyword of K1 is known to be **PALIMPSEST** it is not unreasonable to look at the top left corner of the sculpture text. Combining all these factors lets re-examine the first two rows of cipher text for K1.

E	M	U	F	P	H	Z	L	R	F	A	X	Y	U	S	D	J	K	Z	L	D	K	R	N	S	H	G	N	F	I	V	J	
Y	Q	T	Q	U	X	Q	B	Q	V	Y	U	V	L	L	T	R	E	V	J	Y	Q	T	M	K	Y	R	D	M	F	D		

We can clearly see the word **FUME** written backwards before an alternating sequence of 9 and 10 letter skips give us a resulting string of **PUNYY**. This string is particularly interesting as using the exact same Quagmire III technique as used for the decryptions of K1 and K2, the **encryption** (note, not the decryption!) of **UNYY** with the first four letters of K4 (which are **OBKR**) gives the encrypted text **KRYP**. Something that was initially overlooked is that the first P is attached to a word that is written backward. The last letter of K4 is R, so if we imagine K4 as being a repeating cycle rather than a fixed 97 letter string then this would be the first

letter of K4 if we were to move back by one position from U. With this in mind if we repeat the steps above using **PUNYY** and **ROBKR** then we arrive at an encrypted text of **TKRYP**.

For reasons of simplicity I presented these clues first to show how the linkage between the cryptic clues and the embedded patterns in Kryptos work together to show meaningful results. These clues however were not the ones used to make the initial break. It is in fact possible to arrive at the exact same conclusion using a completely different cryptic clue.

Let's examine the plaintext of K1 again in more detail.

BETWEEN SUBTLE SHADING AND THE ABSENCE OF LIGHT LIES THE NUANCE OF IQLUSION

Let's take this clue at face value and look for the nuances between 'SUBTLE SHADING' and the 'ABSENCE OF LIGHT'. To make this more explicit, let's write them one above the other.

SUBTLE SHADING	SUBTLE SHADING
ABSENCE OF LIGHT	ABSENCE OF LIGHT

Removing the word LIGHT from both lines gives.

SUBE SADN	SUBE SADN
ABSENCE OF	ABSENCE OF

Removing the common letters in both rows to show the subtle difference (the nuance) gives.

USD
CEOF

Let's examine the first two rows of both the K1 cipher text and the K1 plain text again in respect of this result.

E	M	U	F	P	H	Z	L	R	F	A	X	Y	U	S	D	J	K	Z	L	D	K	R	N	S	H	G	N	F	I	V	J	
Y	Q	T	Q	U	X	Q	B	Q	V	Y	U	V	L	L	T	R	E	V	J	Y	Q	T	M	K	Y	R	D	M	F	D		

B	E	T	W	E	E	N	S	U	B	T	L	E	S	H	A	D	I	N	G	A	N	D	T	H	E	A	B	S	E	N	C	
E	O	F	L	I	G	H	T	L	I	E	S	T	H	E	N	U	A	N	C	E	O	F	I	Q	L	U	S	I	O	N		

At first this result may look unremarkable or perhaps even coincidental, but by making one significant change we can achieve a meaningful insight. Before this is shown it is worth reminding ourselves of another clue given in K3 to show that this step is working to instructions rather than a purely creative leap.

THE REMAINS OF PASSAGE DEBRIS THAT ENCUMBERED THE LOWER PART OF THE DOORWAY WAS REMOVED

If the letter U represents the first letter to be paired with the first letter of K4, then this is effectively our starting point or doorway. Any letters prior to this point may be considered debris that are encumbering the lower part of the doorway. Let's extract these letters and pull the remaining text to the left and then place the extracted letters in the void that was vacated on the right (letters that have been moved are shown shaded on the right). Let's repeat this operation for the plain text.

U	S	D	J	K	Z	L	D	K	R	N	S	H	G	N	F	I	V	J	E	M	U	F	P	H	Z	L	R	F	A	X	Y
Y	Q	T	Q	U	X	Q	B	Q	V	Y	U	V	L	L	T	R	E	V	J	Y	Q	T	M	K	Y	R	D	M	F	D	

S	H	A	D	I	N	G	A	N	D	T	H	E	A	B	S	E	N	C	B	E	T	W	E	E	N	S	U	B	T	L	E
E	O	F	L	I	G	H	T	L	I	E	S	T	H	E	N	U	A	N	C	E	O	F	I	O	L	U	S	I	O	N	

This single manipulation of the cipher and plain texts now draws our attention to a new pattern.

U	S	D	J	K	Z	L	D	K	R	N	S	H	G	N	F	I	V	J	E	M	U	F	P	H	Z	L	R	F	A	X	Y
Y	Q	T	Q	U	X	Q	B	Q	V	Y	U	V	L	L	T	R	E	V	J	Y	Q	T	M	K	Y	R	D	M	F	D	

S	H	A	D	I	N	G	A	N	D	T	H	E	A	B	S	E	N	C	B	E	T	W	E	E	N	S	U	B	T	L	E	
E	O	F	L	I	G	H	T	L	I	E	S	T	H	E	N	U	A	N	C	E	O	F	I	O	L	U	S	I	O	N		

The repeating skip sequences of 10 and 9 characters can now be seen to terminating at the boundary delineated by the CEOF marker.

The significance of this repeating pattern every 19 characters is remarkable for a number of reasons, which will be touched upon later, but probably of most importance at this juncture is the observation that the XY string now terminates the line. This is believed to be significant as the X plane is known mathematically as the Abscissa and the Y plane as the Ordinate, so the top right of the sculpture now seems to imply a definite X and Y axis.

The next clue we are given that is complementary to this rearrangement comes from re-using the same clue as before, which is **HOT AIR ESCAPING**. Let's take a look at the first three lines of the plain and cipher text under this re-arrangement.

U	S	D	J	K	Z	L	D	K	R	N	S	H	G	N	F	I	V	J	E	M	U	F	P	H	Z	L	R	F	A	X	Y	
Y	Q	T	Q	U	X	Q	B	Q	V	Y	U	V	L	L	T	R	E	V	J	Y	Q	T	M	K	Y	R	D	M	F	D		
V	F	P	J	U	D	E	E	H	Z	W	E	T	Z	Y	V	G	W	H	K	K	O	E	T	G	F	O	J	N	C	E		

S	H	A	D	I	N	G	A	N	D	T	H	E	A	B	S	E	N	C	B	E	T	W	E	E	N	S	U	B	T	L	E	
E	O	F	L	I	G	H	T	L	I	E	S	T	H	E	N	U	A	N	C	E	O	F	I	Q	L	U	S	I	O	N		
I	T	W	A	S	T	O	T	A	L	L	Y	I	N	V	I	S	I	B	L	E	H	O	W	S	T	H	A	T	P	O		

It can be seen quite clearly that we have two distinct sections of the plain text. One has a repeating series of 19 characters and the other comprises what at first seems to be irregular row lengths. Both sections contain the word HOT in the vertical plane. The direction of each being reversed. Another important point to note here is that each of the strings forming the word HOT in the vertical plane are crossed by the letters EOF forming a cross.

S	H	A	D	I	N	G	A	N	D	T	H	E	A	B	S	E	N	C	B	E	T	W	E	E	N	S	U	B	T	L	E	
E	O	F	L	I	G	H	T	L	I	E	S	T	H	E	N	U	A	N	C	E	O	F	I	Q	L	U	S	I	O	N		
I	T	W	A	S	T	O	T	A	L	L	Y	I	N	V	I	S	I	B	L	E	H	O	W	S	T	H	A	T	P	O		

If this is still seen to be a coincidence then let's do the math with the repeating rows of 19 characters. Two rows of 19 are 38 and three rows are 57, which are the hours and minutes of the first coordinates given in K2 for the location of the Kryptos sculpture.

Let's remind ourselves again of those coordinates.

THIRTY EIGHT DEGREES FIFTY SEVEN MINUTES SIX POINT FIVE SECONDS NORTH SEVENTY SEVEN DEGREES EIGHT MINUTES FORTY FOUR SECONDS WEST

With this in mind can we push the scheme given above any further?

U	S	D	J	K	Z	L	D	K	R	N	S	H	G	N	F	I	V	J	E	M	U	F	P	H	Z	L	R	F	A	X	Y	
Y	Q	T	Q	U	X	Q	B	Q	V	Y	U	V	L	L	T	R	E	V	J	Y	Q	T	M	K	Y	R	D	M	F	D		
V	F	P	J	U	D	E	E	H	Z	W	E	T	Z	Y	V	G	W	H	K	K	Q	E	T	G	F	Q	J	N	C	E		
G	G	W	H	K	K	?	D	Q	M	C	P	F	Q	Z	D	Q	M	M	I	A	G	P	F	X	H	Q	R	L	G			

To complete the first set of coordinates we are obviously looking for a representation of the number 6.5. The fourth line shows us 6 letters before the question mark, so if we were to take another five after the question mark, assuming that it is somehow indicative of a decimal point then we would be left with 7 spare characters as shown below.

U	S	D	J	K	Z	L	D	K	R	N	S	H	G	N	F	I	V	J	E	M	U	F	P	H	Z	L	R	F	A	X	Y	
Y	Q	T	Q	U	X	Q	B	Q	V	Y	U	V	L	L	T	R	E	V	J	Y	Q	T	M	K	Y	R	D	M	F	D		
V	F	P	J	U	D	E	E	H	Z	W	E	T	Z	Y	V	G	W	H	K	K	Q	E	T	G	F	Q	J	N	C	E		
G	G	W	H	K	K	?	D	Q	M	C	P	F	Q	Z	D	Q	M	M	I	A	G	P	F	X	H	Q	R	L	G			

Before explaining the relevance of this assertion, let's quickly re-examine the first four rows of the corresponding plain text.

S	H	A	D	I	N	G	A	N	D	T	H	E	A	B	S	E	N	C	B	E	T	W	E	E	N	S	U	B	T	L	E	
E	O	F	L	I	G	H	T	L	I	E	S	T	H	E	N	U	A	N	C	E	O	F	I	Q	L	U	S	I	O	N		
I	T	W	A	S	T	O	T	A	L	L	Y	I	N	V	I	S	I	B	L	E	H	O	W	S	T	H	A	T	P	O		
S	S	I	B	L	E	?	T	H	E	Y	U	S	E	D	T	H	E	E	A	R	T	H	S	M	A	G	N	E	T			
I	C	F	I	E	L	D	X	T	H	E	I	N	F	O	R	M	A	T	I	O	N	W	A	S	G	A	T	H	E	R		

We can now see that we have four rows that encode all the coordinates of the latitude given in K2. The remaining seven letters will be explained away in the section, but this is a good time to go off road slightly and describe a phenomenon that may have implications for the correct ordering of the letters. So far we have only made one alteration, which was to move 13 letters from the begging of row one to the end. If we look in the vertical plane we can see that the string HOT has been extended in both instances to encompass an anagram of the words SOUTH and NORTH that is only deficient in an U and R respectively. We can see this pattern repeated again in the left and right hand blocks respectively with the polarities reversed. Interestingly this would be completely obvious for the SOUTH anagram on the right hand side were it not for the double U in UNDERGRUUND, leading one to suspect it was obfuscated deliberately. The reasoning for this is no doubt in order to make us realise that the one on the left is the one that we should eventually be focused on. It can also be seen that anagrams of EAST and WEST also missing one letter can also be found. These patterns are deemed to be relevant, so will be revisited in a later section, but for now the purpose is purely to show that we could make further structural changes to the row and column orderings in order to line these strings up in the vertical plane as shown below.

S	H	A	D	I	N	G	A	N	D	T	H	E	A	B	S	E	N	C	B	E	T	W	E	E	N	S	U	B	T	L	E	
E	O	F	L	I	G	H	T	L	I	E	S	T	H	E	N	U	A	N	C	E	O	F	I	Q	L	U	S	I	O	N		
I	T	W	A	S	T	O	T	A	L	L	Y	I	N	V	I	S	I	B	L	E	H	O	W	S	T	H	A	T	P	O		
S	S	I	B	L	E	?	T	H	E	Y	U	S	E	D	T	H	E	E	A	R	T	H	S	M	A	G	N	E	T			
I	C	F	I	E	L	D	X	T	H	E	I	N	F	O	R	M	A	T	I	O	N	W	A	S	G	A	T	H	E	R		
E	D	A	N	D	T	R	A	N	S	M	I	T	T	E	D	U	N	D	E	R	G	R	U	U	N	D	T	O	A	N	U	
N	K	N	O	W	N	L	O	C	A	T	I	O	N	X	D	O	E	S	L	A	N	G	L	E	Y	K	N	O	W	A		
B	O	U	T	T	H	I	S	?	T	H	E	Y	S	H	O	U	L	D	I	T	S	B	U	R	I	E	D	O	U	T		
T	H	E	R	E	S	O	M	E	W	H	E	R	E	X	W	H	O	K	N	O	W	S	T	H	E	E	X	A	C	T	L	
O	C	A	T	I	O	N	?	O	N	L	Y	W	W	T	H	I	S	W	A	S	H	I	S	L	A	S	T	M	E	S		
S	A	G	E	X	T	H	I	R	T	Y	E	I	G	H	T	D	E	G	R	E	E	S	F	I	F	T	Y	S	E			
V	E	N	M	I	N	U	T	E	S	S	I	X	P	O	I	N	T	F	I	V	E	S	E	C	O	N	D	S	N	O		
R	T	H	S	E	V	E	N	T	Y	S	E	V	E	N	D	E	G	R	E	E	S	E	I	G	H	T	M	I	N	U		
T	E	S	F	O	R	T	Y	F	O	U	R	S	E	C	O	N	D	S	W	E	S	T	X	L	A	Y	E	R	T	W	O	

After column reorder in the relevant rows we get the following.

S	H	A	D	I	N	G	A	N	D	T	H	E	A	B	S	E	N	C	B	E	T	W	E	E	N	S	U	B	T	L	E	
E	O	F	L	I	G	H	T	L	I	E	S	T	H	E	N	U	A	N	C	E	O	F	I	Q	L	U	S	I	O	N		
I	T	W	A	S	T	O	T	A	L	L	Y	I	N	V	I	S	I	B	L	E	H	O	W	S	T	H	A	T	P	O		
S	S	I	B	L	E	?	T	H	E	Y	U	S	E	D	T	H	E	E	A	R	T	H	S	M	A	G	N	E	T			
I	C	F	I	E	L	D	X	T	H	E	I	N	F	O	R	M	A	T	I	O	N	W	A	S	G	A	T	H	E	R		
A	N	D	T	R	A	N	S	M	I	T	T	E	D	U	N	D	E	D	G	R	U	U	N	D	T	O	A	N	U	E	R	
N	O	W	N	L	O	C	A	T	I	O	N	X	D	O	E	S	N	K	N	G	L	E	Y	K	N	O	W	A	L	A		
U	T	T	H	I	S	?	T	H	E	Y	S	H	O	U	L	D	B	O	S	B	U	R	I	E	D	O	U	T	I	T		
E	R	E	S	O	M	E	W	H	E	R	E	X	W	H	O	K	T	H	W	S	T	H	E	E	X	A	C	T	L	N	O	
A	T	I	O	N	?	O	N	L	Y	W	W	T	H	I	S	W	O	C	H	I	S	L	A	S	T	M	E	S	A	S		
S	A	G	E	X	T	H	I	R	T	Y	E	I	G	H	T	D	E	G	E	E	S	F	I	F	T	Y	S	E	R			
V	E	N	M	I	N	U	T	E	S	S	I	X	P	O	I	N	T	F	V	E	S	E	C	O	N	D	S	N	O	I		
R	T	H	S	E	V	E	N	T	Y	S	E	V	E	N	D	E	G	R	E	S	E	I	G	H	T	M	I	N	U	E		
T	E	S	F	O	R	T	Y	F	O	U	R	S	E	C	O	N	D	S	E	S	T	X	L	A	Y	E	R	T	W	O	W	

Before moving on to the next section it's important to note that when we look at the corresponding cipher text that UNYY is on the first 2 rows of 19 wrapped around a running

key of what is supposed to be the word SOUTH. If we look in the third row of 19 wrapped around what is supposed to be NORTH then we have KC or CK. The string UNYYCK when **encrypted** against OBKRUE (the first six letters of K4) gives KRYPTO. So to be clear, the first two rows coding for 38 give us the encrypted string KRYPT and the next row down coding for 57 gives us the encrypted string TO.

U	S	D	J	K	Z	L	D	K	R	N	S	H	G	N	F	I	V	J	E	M	U	F	P	H	Z	L	R	F	A	X	Y	
Y	Q	T	Q	U	X	Q	B	Q	V	Y	U	V	L	L	T	R	E	V	J	Y	Q	T	M	K	Y	R	D	M	F	D		
V	F	P	J	U	D	E	E	H	Z	W	E	T	Z	Y	V	G	W	H	K	K	Q	E	T	G	F	Q	J	N	C	E		
G	G	W	H	K	K	?	D	Q	M	C	P	F	Q	Z	D	Q	M	M	I	A	G	P	F	X	H	Q	R	L	G			
T	I	M	V	M	Z	J	A	N	Q	L	V	K	Q	E	D	A	G	D	V	F	R	P	J	U	N	G	E	U	N	A		

An important question to ask at each step is whether we have any clues given in the plain text that support such a scheme. In this instance there is a strong indication given that we're on the right track given by the following clues given in the K2 plain text.

IT WAS TOTALLY INVISIBLE HOWS THAT POSSIBLE? THEY USED THE EARTH'S MAGNETIC FIELD.

Two aspects of this clue are important, the first is that EARTHSMAGNETICFIELD is a string of 19 characters and secondly that to make a scheme such as the one above INVISIBLE requires the use of a cryptographic technique called Steganography.

Part III

Steganography

Before embarking on this next section we need to clarify an important point. The grid system that the Kryptos letters have been put into is an abstraction, just as geometry is an abstraction of the real world. The individual blocks making the grid have a number value just like coordinates on an X and Y axis. To give an example, three grid blocks means the number three irrespective of the letters that may be in each box. I make this point as we have already made a manipulation of the text, but this has no bearing on the steganography that has been used in Kryptos, so for clarity I shall show the number surface using the original layout rather than with the re-arrangement of the first row used in the previous section.

This abstraction is a powerful one as we shall see that the numerical values associated with the grid system can be thought of as an analogue of the modern day hardware model. The letters themselves can be seen as being data that fill the registers of this hardware and our decryption technique can be thought of as an algorithm that is run using the underlying mechanics of this machine. This may sound confusing at first given that Kryptos is purely a series of letters printed on copper, but once explained, the underlying genius of the sculpture layout will become evident.

Let's fill in what we currently have in terms of embedded coordinates from the last section.

E	M	U	F	P	H	Z	L	R	F	A	X	Y	U	S	D	J	K	Z	L	D	K	R	N	S	H	G	N	F	I	V	J		
Y	Q	T	Q	U	X	Q	B	Q	V	Y	U	V	L	L	T	R	E	V	J	Y	Q	T	M	K	Y	R	D	M	F	D			
V	F	P	J	U	D	E	E	H	Z	W	E	T	Z	Y	V	G	W	H	K	K	Q	E	T	G	F	Q	J	N	C	E			
G	G	W	H	K	K	?	D	Q	M	C	P	F	Q	Z	D	Q	M	M	I	A	G	P	F	X	H	Q	R	L	G				

The number of boxes in rows one and two give us 38. The addition of the next row gives us 57. The next row is 6 followed by a question mark and then 5. So using the exact same technique we now want to find the second set of coordinates for the longitude, which are 77, 8 and 44 respectively. Here we have our block of 77 and our block of 8 terminated by the question mark. The blank rows above the row of 8 and to the left of the 77 number 36 in total.

E	M	U	F	P	H	Z	L	R	F	A	X	Y	U	S	D	J	K	Z	L	D	K	R	N	S	H	G	N	F	I	V	J	
Y	Q	T	Q	U	X	Q	B	Q	V	Y	U	V	L	L	T	R	E	V	J	Y	Q	T	M	K	Y	R	D	M	F	D		
V	F	P	J	U	D	E	E	H	Z	W	E	T	Z	Y	V	G	W	H	K	K	Q	E	T	G	F	Q	J	N	C	E		
G	G	W	H	K	K	?	D	Q	M	C	P	F	Q	Z	D	Q	M	M	I	A	G	P	F	X	H	Q	R	L	G			
T	I	M	V	M	Z	J	A	N	Q	L	V	K	Q	E	D	A	G	D	V	F	R	P	J	U	N	G	E	U	N	A		
Q	Z	G	Z	L	E	C	G	Y	U	X	U	E	E	N	J	T	B	J	L	B	Q	C	R	T	B	J	D	F	H	R	R	
Y	I	Z	E	T	K	Z	E	M	V	D	U	F	K	S	J	H	K	F	W	H	K	U	W	Q	L	S	Z	F	T	I		
H	H	D	D	D	U	V	H	?	D	W	K	B	F	U	F	P	W	N	T	D	F	I	Y	C	U	Q	Z	E	R	E		
E	V	L	D	K	F	E	Z	M	O	Q	Q	J	L	T	T	U	G	S	Y	Q	P	F	E	U	N	L	A	V	I	D	X	
F	L	G	G	T	E	Z	?	F	K	Z	B	S	F	D	Q	V	G	O	G	I	P	U	F	X	H	H	D	R	K	F		
F	H	Q	N	T	G	P	U	A	E	C	N	U	V	P	D	J	M	Q	C	L	Q	U	M	U	N	E	D	F	Q			
E	L	Z	Z	V	R	R	G	K	F	F	V	O	E	E	X	B	D	M	V	P	N	F	Q	X	E	Z	L	G	R	E		
D	N	Q	F	M	P	N	Z	G	L	F	L	F	M	R	J	Q	Y	A	L	M	G	N	U	V	P	D	X	V	K	P		
D	O	U	M	E	B	E	D	M	H	D	A	F	M	J	G	Z	N	U	P	L	G	E	W	J	L	L	A	E	T	G		

By using the same method we used earlier (of addition to the existing coordinates) then we arrive at our required 44 blocks by adding the 8. ***Therefore we now see that the entire set of coordinates given in K2 are embedded completely within the top left of the sculptures letter layout irrespective of the letters themselves.***

E	M	U	F	P	H	Z	L	R	F	A	X	Y	U	S	D	J	K	Z	L	D	K	R	N	S	H	G	N	F	I	V	J		
Y	Q	T	Q	U	X	Q	B	Q	V	Y	U	V	L	L	T	R	E	V	J	Y	Q	T	M	K	Y	R	D	M	F	D			
V	F	P	J	U	D	E	E	H	Z	W	E	T	Z	Y	V	G	W	H	K	K	Q	E	T	G	F	Q	J	N	C	E			
G	G	W	H	K	K	?	D	Q	M	C	P	F	Q	Z	D	Q	M	M	I	A	G	P	F	X	H	Q	R	L	G				
T	I	M	V	M	Z	J	A	N	Q	L	V	K	Q	E	D	A	G	D	V	F	R	P	J	U	N	G	E	U	N	A			
Q	Z	G	Z	L	E	C	G	Y	U	X	U	E	E	N	J	T	B	J	L	B	Q	C	R	T	B	J	D	F	H	R	R		
Y	I	Z	E	T	K	Z	E	M	V	D	U	F	K	S	J	H	K	F	W	H	K	U	W	Q	L	S	Z	F	T	I			
H	H	D	D	D	U	V	H	?	D	W	K	B	F	U	F	P	W	N	T	D	F	I	Y	C	U	Q	Z	E	R	E			
E	V	L	D	K	F	E	Z	M	O	Q	Q	J	L	T	T	U	G	S	Y	Q	P	F	E	U	N	L	A	V	I	D	X		
F	L	G	G	T	E	Z	?	F	K	Z	B	S	F	D	Q	V	G	O	G	I	P	U	F	X	H	H	D	R	K	F			
F	H	Q	N	T	G	P	U	A	E	C	N	U	V	P	D	J	M	Q	C	L	Q	U	M	U	N	E	D	F	Q				
E	L	Z	Z	V	R	R	G	K	F	F	V	O	E	E	X	B	D	M	V	P	N	F	Q	X	E	Z	L	G	R	E			
D	N	Q	F	M	P	N	Z	G	L	F	L	P	M	R	J	Q	Y	A	L	M	G	N	U	V	P	D	X	V	K	P			
D	Q	U	M	E	B	E	D	M	H	D	A	F	M	J	G	Z	N	U	P	L	G	E	W	J	L	L	A	E	T	G			

What now stands out is that the rows and question marks are meaningful in terms of defining numbers that have significance. This finding leads to remarkable result when we examine the next three rows of the sculpture.

H	H	D	D	D	U	V	H	?	D	W	K	B	F	U	F	P	W	N	T	D	F	I	Y	C	U	Q	Z	E	R	E		
E	V	L	D	K	F	E	Z	M	O	Q	Q	J	L	T	T	U	G	S	Y	Q	P	F	E	U	N	L	A	V	I	D	X	
F	L	G	G	T	E	Z	?	F	K	Z	B	S	F	D	Q	V	G	O	G	I	P	U	F	X	H	H	D	R	K	F		
F	H	Q	N	T	G	P	U	A	E	C	N	U	V	P	D	J	M	Q	C	L	Q	U	M	U	N	E	D	F	Q			

We can see that we have blocks in these rows of 8, 3, 12, 7 and 4 respectively. The blocks of 3, 12, and 7 give us a result that seems to be far from coincidence. Using the same methods of addition that we used previously we can add 7 to 12 to give us 19. This doesn't sound any more remarkable than being another incident of the number 19 in the row scheme, but on closer inspection we can see that 19 divided by 7 is a close approximation to Euler's number otherwise known as e or 2.718. To show that this is more than chance lets now add the 3 to the accumulated value of 19. We can immediately see that we have another well known ratio of 22 and 7, in other words an approximation to π or 3.14. To see e and π in such close proximity immediately conjures up the relations inherent in Euler's Identity. For anyone unfamiliar with this identity it is sometimes referred to as one of the most beautiful equations in mathematics as it forms a link between 5 mathematical fundamentals, which are π , e, 1, 0 and the imaginary unit of i (the square root of -1). The identity is given in full as:

$$e^{i\pi} + 1 = 0$$

To complete our initial observations of the number patterns we can see that the block of four letters is immediately followed by four rows of 12 letters making a block of 48 and 4 or 52 in total.

E	M	U	F	P	H	Z	L	R	F	A	X	Y	U	S	D	J	K	Z	L	D	K	R	N	S	H	G	N	F	I	V	J		
Y	Q	T	Q	U	X	Q	B	Q	V	Y	U	V	L	L	T	R	E	V	J	Y	Q	T	M	K	Y	R	D	M	F	D			
V	F	P	J	U	D	E	E	H	Z	W	E	T	Z	Y	V	G	W	H	K	K	Q	E	T	G	F	Q	J	N	C	E			
G	G	W	H	K	K	?	D	Q	M	C	P	F	Q	Z	D	Q	M	M	I	A	G	P	F	X	H	Q	R	L	G				
T	I	M	V	M	Z	J	A	N	Q	L	V	K	Q	E	D	A	G	D	V	F	R	P	J	U	N	G	E	U	N	A			
Q	Z	G	Z	L	E	C	G	Y	U	X	U	E	E	N	J	T	B	J	L	B	Q	C	R	T	B	J	D	F	H	R	R		
Y	I	Z	E	T	K	Z	E	M	V	D	U	F	K	S	J	H	K	F	W	H	K	U	W	Q	L	S	Z	F	T	I			
H	H	D	D	D	U	V	H	?	D	W	K	B	F	U	F	P	W	N	T	D	F	I	Y	C	U	Q	Z	E	R	E			
E	V	L	D	K	F	E	Z	M	O	Q	Q	J	L	T	T	U	G	S	Y	Q	P	F	E	U	N	L	A	V	I	D	X		
F	L	G	G	T	E	Z	?	F	K	Z	B	S	F	D	Q	V	G	O	G	I	P	U	F	X	H	H	D	R	K	F			
F	H	Q	N	T	G	P	U	A	E	C	N	U	V	P	D	J	M	Q	C	L	Q	U	M	U	N	E	D	F	Q				
E	L	Z	Z	V	R	R	G	K	F	F	V	O	E	E	X	B	D	M	V	P	N	F	Q	X	E	Z	L	G	R	E			
D	N	Q	F	M	P	N	Z	G	L	F	L	P	M	R	J	Q	Y	A	L	M	G	N	U	V	P	D	X	V	K	P			
D	Q	U	M	E	B	E	D	M	H	D	A	F	M	J	G	Z	N	U	P	L	G	E	W	J	L	L	A	E	T	G			

A figure of 52 is interesting as it immediately makes us think of calendar weeks. With this in mind let's now take a look at the right hand side of the letter layout.

E	M	U	F	P	H	Z	L	R	F	A	X	Y	U	S	D	J	K	Z	L	D	K	R	N	S	H	G	N	F	I	V	J	
Y	Q	T	Q	U	X	Q	B	Q	V	Y	U	V	L	L	T	R	E	V	J	Y	Q	T	M	K	Y	R	D	M	F	D		
V	F	P	J	U	D	E	E	H	Z	W	E	T	Z	Y	V	G	W	H	K	K	Q	E	T	G	F	Q	J	N	C	E		
G	G	W	H	K	K	?	D	Q	M	C	P	F	Q	Z	D	Q	M	M	I	A	G	P	F	X	H	Q	R	L	G			
T	I	M	V	M	Z	J	A	N	Q	L	V	K	Q	E	D	A	G	D	V	F	R	P	J	U	N	G	E	U	N	A		
Q	Z	G	Z	L	E	C	G	Y	U	X	U	E	E	N	J	T	B	J	L	B	Q	C	R	T	B	J	D	F	H	R	R	
Y	I	Z	E	T	K	Z	E	M	V	D	U	F	K	S	J	H	K	F	W	H	K	U	W	Q	L	S	Z	F	T	I		
H	H	D	D	D	U	V	H	?	D	W	K	B	F	U	F	P	W	N	T	D	F	I	Y	C	U	Q	Z	E	R	E		
E	V	L	D	K	F	E	Z	M	O	Q	Q	J	L	T	T	U	G	S	Y	Q	P	F	E	U	N	L	A	V	I	D	X	
F	L	G	G	T	E	Z	?	F	K	Z	B	S	F	D	Q	V	G	O	G	I	P	U	F	X	H	H	D	R	K	F		
F	H	Q	N	T	G	P	U	A	E	C	N	U	V	P	D	J	M	Q	C	L	Q	U	M	U	N	E	D	F	Q			
E	L	Z	Z	V	R	R	G	K	F	F	V	O	E	E	X	B	D	M	V	P	N	F	Q	X	E	Z	L	G	R	E		
D	N	Q	F	M	P	N	Z	G	L	F	L	P	M	R	J	Q	Y	A	L	M	G	N	U	V	P	D	X	V	K	P		
D	Q	U	M	E	B	E	D	M	H	D	A	F	M	J	G	Z	N	U	P	L	G	E	W	J	L	L	A	E	T	G		

E	N	D	Y	A	H	R	O	H	N	L	S	R	H	E	O	C	P	T	E	O	I	B	I	D	Y	S	H	N	A	I	A		
C	H	T	N	R	E	Y	U	L	D	S	L	L	S	L	L	N	O	H	S	N	O	S	M	R	W	X	M	N	E				
T	P	R	N	G	A	T	I	H	N	R	A	R	P	E	S	L	N	N	E	L	E	B	L	P	I	I	A	C	A	E			
W	M	T	W	N	D	I	T	E	E	N	R	A	H	C	T	E	N	E	U	D	R	E	T	N	H	A	E	O	E				
T	F	O	L	S	E	D	T	I	W	E	N	H	A	E	I	O	Y	T	E	Y	Q	H	E	E	N	C	T	A	Y	C	R		
E	I	F	T	B	E	R	S	P	A	M	H	H	E	W	E	N	A	E	A	T	A	M	A	T	E	G	Y	E	E	R	L	B	
T	E	E	F	O	A	S	F	I	O	T	U	E	T	U	A	E	O	T	O	A	R	M	A	E	E	R	T	N	R	T	I		
B	S	E	D	D	N	I	A	A	H	T	T	M	S	T	E	W	P	I	E	R	O	A	G	R	I	E	W	F	E	B			
A	E	C	T	D	D	H	I	L	C	E	I	H	S	I	T	E	G	O	E	A	O	S	D	D	R	Y	D	L	O	R	I	T	
R	K	L	M	L	E	H	A	G	T	D	H	A	R	D	P	N	E	O	H	M	G	F	M	F	E	U	H	E					
E	C	D	M	R	I	P	F	E	I	M	E	H	N	L	S	S	T	T	R	T	V	D	O	H	W	?	O	B	K	R			
U	O	X	O	G	H	U	L	B	S	O	L	I	F	B	B	W	F	L	R	V	Q	Q	P	R	N	G	K	S	S	O			
T	W	T	Q	S	J	Q	S	S	E	K	Z	Z	W	A	T	J	K	L	U	D	I	A	W	I	N	F	B	N	Y	P			
V	T	T	M	Z	F	P	K	W	G	D	K	Z	X	T	J	C	D	I	G	K	U	H	U	A	U	E	K	C	A	R			

If it is not immediately obvious then the green block on the right hand side consists of 28 rows of 13 blocks making a total of 364. The rather out of place looking T that sits outside of the grid brings us to a grand total of 365. Let's also not forget that the scheme above does

not include the 'missed X' in the 14th row, so when added (every leap year) we have a total of 366 squares present on the right hand half of the sculpture. At this point it also becomes apparent that the spaces are important as there are 28 of them, which shows our first potential connection to something relating to the lunar calendar. From now on we shall show the spaces with an asterisk in order to complete the rows. I will also show the effect of adding the 'missed X' in this next diagram to show the grid in its 366 square leap year mode.

E	M	U	F	P	H	Z	L	R	F	A	X	Y	U	S	D	J	K	Z	L	D	K	R	N	S	H	G	N	F	I	V	J	
Y	Q	T	Q	U	X	Q	B	Q	V	Y	U	V	L	L	T	R	E	V	J	Y	Q	T	M	K	Y	R	D	M	F	D	*	
V	F	P	J	U	D	E	E	H	Z	W	E	T	Z	Y	V	G	W	H	K	K	Q	E	T	G	F	Q	J	N	C	E	*	
G	G	W	H	K	K	?	D	Q	M	C	P	F	Q	Z	D	Q	M	I	A	G	P	F	X	H	Q	R	L	G	*	*		
T	I	M	V	M	Z	J	A	N	Q	L	V	K	Q	E	D	A	G	D	V	F	R	P	J	U	N	G	E	U	N	A	*	
Q	Z	G	Z	L	E	C	G	Y	U	X	U	E	E	N	J	T	B	J	L	B	Q	C	R	T	B	J	D	F	H	R	R	
Y	I	Z	E	T	K	Z	E	M	V	D	U	F	K	S	J	H	K	F	W	H	K	U	W	Q	L	S	Z	F	T	I	*	
H	H	D	D	D	U	V	H	?	D	W	K	B	F	U	F	P	W	N	T	D	F	I	Y	C	U	Q	Z	E	R	E	*	
E	V	L	D	K	F	E	Z	M	O	Q	Q	J	L	T	T	U	G	S	Y	Q	P	F	E	U	N	L	A	V	I	D	X	
F	L	G	G	T	E	Z	?	F	K	Z	B	S	F	D	Q	V	G	O	G	I	P	U	F	X	H	H	D	R	K	F	*	
F	H	Q	N	T	G	P	U	A	E	C	N	U	V	P	D	J	M	Q	C	L	Q	U	M	U	N	E	D	F	Q	*	*	
E	L	Z	Z	V	R	R	G	K	F	F	V	O	E	E	X	B	D	M	V	P	N	F	Q	X	E	Z	L	G	R	E	*	
D	N	Q	F	M	P	N	Z	G	L	F	L	P	M	R	J	Q	Y	A	L	M	G	N	U	V	P	D	X	V	K	P	*	
D	Q	U	M	E	B	E	D	M	H	D	A	F	M	J	G	Z	N	U	P	L	G	E	S	W	J	L	L	A	E	T	G	*

E	N	D	Y	A	H	R	O	H	N	L	S	R	H	E	O	C	P	T	E	O	I	B	I	D	Y	S	H	N	A	I	A	
C	H	T	N	R	E	Y	U	L	D	S	L	L	S	L	L	N	O	H	S	N	O	S	M	R	W	X	M	N	E	*	*	
T	P	R	N	G	A	T	I	H	N	R	A	R	P	E	S	L	N	N	E	L	E	B	L	P	I	I	A	C	A	E	*	
W	M	T	W	N	D	I	T	E	E	N	R	A	H	C	T	E	N	E	U	D	R	E	T	N	H	A	E	O	E	*	*	
T	F	O	L	S	E	D	T	I	W	E	N	H	A	E	I	O	Y	T	E	Y	Q	H	E	E	N	C	T	A	Y	C	R	
E	I	F	T	B	R	S	P	A	M	H	H	E	W	E	N	A	T	A	M	A	T	E	G	Y	E	E	R	L	B	*	*	
T	E	E	F	O	A	S	F	I	O	T	U	E	T	U	A	E	O	T	O	A	R	M	A	E	E	R	T	N	R	T	I	
B	S	E	D	D	N	I	A	A	H	T	T	M	S	T	E	W	P	I	E	R	O	A	G	R	I	E	W	F	E	B	*	
A	E	C	T	D	D	H	I	L	C	E	I	H	S	I	T	E	G	O	E	A	O	S	D	D	R	Y	D	L	O	R	I	T
R	K	L	M	L	E	H	A	G	T	D	H	A	R	D	P	N	E	O	H	M	G	F	M	F	E	U	H	E	*	*	*	
E	C	D	M	R	I	P	F	E	I	M	E	H	N	L	S	S	T	T	R	T	V	D	O	H	W	?	O	B	K	R	*	
U	O	X	O	G	H	U	L	B	S	O	L	I	F	B	B	W	F	L	R	V	Q	Q	P	R	N	G	K	S	S	O	*	
T	W	T	Q	S	J	Q	S	S	E	K	Z	Z	W	A	T	J	K	D	L	U	D	I	A	W	I	N	F	B	N	Y	P	*
V	T	T	M	Z	F	P	K	W	G	D	K	Z	X	T	J	C	D	I	G	K	U	H	U	A	U	E	K	C	A	R	*	

If the right hand side of the sculpture represents the year 1990, which was not a leap year then the position of the T that sits outside of the grid represents a date of the 28th October, which interestingly is the date that commemorates the end of the Cuban Missile Crisis. If we use the 366 day scheme (i.e. with the missing X replaced) then we have a date of October 27th. You will see that there are 7 spaces from the T until the end of the grid which represents the 31st December. If we count back to the position of T from the 31st December, but do not include the spaces, then we arrive at a date of November 3rd for the 366 day scheme or November the 2nd for the 365 day scheme. If the reader skipped the introductory section then I'll remind them now that the sculpture was unveiled on the 3rd November 1990 the day after the *previous night's full moon*.

Some additional observations about the calendar section can be made immediately that are of significance. The first is that the extra letter is currently added at a position

corresponding to June 23rd. Interestingly there is a grid distance between the T that sits on the outside of the grid and the position of the X that is added of 127 characters. 127 is a 7 bit binary number realised as the summation of 64, 32, 16, 8, 4, 2 and 1. In the leap year scheme of 366 characters it corresponds to a date of 10 27, which further piques our curiosity given the references in the morse code aspects of the installation to 'digital interpretation'. The calendar rows have been shaded and numbered below to show the distinct month boundaries starting on January 1st in a non leap year.

1	2	3	4	5	6	7	8	9	10	11	12	13	
14	15	16	17	18	19	20	21	22	23	24	25	26	
27	28	29	30	31	1	2	3	4	5	6	7	8	
9	10	11	12	13	14	15	16	17	18	19	20	21	
22	23	24	25	26	27	28	1	2	3	4	5	6	
7	8	9	10	11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30	31	1	2	
3	4	5	6	7	8	9	10	11	12	13	14	15	
16	17	18	19	20	21	22	23	24	25	26	27	28	
29	30	1	2	3	4	5	6	7	8	9	10	11	
12	13	14	15	16	17	18	19	20	21	22	23	24	
25	26	27	28	29	30	31	1	2	3	4	5	6	
7	8	9	10	11	12	13	14	15	16	17	18	19	
20	21	22	23	24	25	26	27	28	29	30	1	2	
3	4	5	6	7	8	9	10	11	12	13	14	15	
16	17	18	19	20	21	22	23	24	25	26	27	28	
29	30	31	1	2	3	4	5	6	7	8	9	10	
11	12	13	14	15	16	17	18	19	20	21	22	23	
24	25	26	27	28	29	30	31	1	2	3	4	5	
6	7	8	9	10	11	12	13	14	15	16	17	18	
19	20	21	22	23	24	25	26	27	28	29	30	1	
2	3	4	5	6	7	8	9	10	11	12	13	14	
15	16	17	18	19	20	21	22	23	24	25	26	27	28
29	30	31	1	2	3	4	5	6	7	8	9		
10	11	12	13	14	15	16	17	18	19	20	21	22	
23	24	25	26	27	28	29	30	1	2	3	4	5	
6	7	8	9	10	11	12	13	14	15	16	17	18	
19	20	21	22	23	24	25	26	27	28	29	30	31	

We saw earlier that the North and South anagrams appeared to be compelling us to re-arrange the data in some way and again here we feel an almost automatic compulsion to re-arrange the data so that the position of the missing X naturally assumes the position of February 29th. First lets re-arrange the date ordering of the underlying grid to see what our start position value now becomes if we shift Feb 29th forward 114 places to occupy the position of the reinstated X. This re-arrangement is shown below, but my own feelings about any significance of the date will remain off book as I want to concentrate on the hard facts around the numbers as opposed to possible inferred meanings of significance in this particular paper. The reader is at liberty of course to shift the cipher and plaintext letters themselves forward to conform to the same pattern to see if any additional information is extractable from such a re-arrangement. The dates that now sit outside the grid are the 9th March and 5th July respectively. In the previous scheme for a leap year the dates would have been the 3rd of July and 27th October.

9	10	11	12	13	14	15	16	17	18	19	20	21	
22	23	24	25	26	27	28	29	30	1	2	3	4	
5	6	7	8	9	10	11	12	13	14	15	16	17	
18	19	20	21	22	23	24	25	26	27	28	29	30	
31	1	2	3	4	5	6	7	8	9	10	11	12	
13	14	15	16	17	18	19	20	21	22	23	24	25	
26	27	28	29	30	1	2	3	4	5	6	7	8	
9	10	11	12	13	14	15	16	17	18	19	20	21	
22	23	24	25	26	27	28	29	30	31	1	2	3	
4	5	6	7	8	9	10	11	12	13	14	15	16	
17	18	19	20	21	22	23	24	25	26	27	28	29	
30	31	1	2	3	4	5	6	7	8	9	10	11	
12	13	14	15	18	17	18	19	20	21	22	23	24	
25	26	27	28	29	1	2	3	4	5	6	7	8	9
10	11	12	13	14	15	16	17	18	19	20	21	22	
23	24	25	26	27	28	29	30	31	1	2	3	4	
5	6	7	8	9	10	11	12	13	14	15	16	17	
18	19	20	21	22	23	24	25	26	27	28	29	30	
1	2	3	4	5	6	7	8	9	10	11	12	13	
14	15	16	17	18	19	20	21	22	23	24	25	26	
27	28	29	30	31	1	2	3	4	5	6	7	8	
9	10	11	12	13	14	15	16	17	18	19	20	21	
22	23	24	25	26	27	28	29	30	1	2	3	4	5
6	7	8	9	10	11	12	13	14	15	16	17	18	
19	20	21	22	23	24	25	26	27	28	29	30	31	
1	2	3	4	5	6	7	8	9	10	11	12	13	
14	15	16	17	18	19	20	21	22	23	24	25	26	
27	28	29	30	31	1	2	3	4	5	6	7	8	

The leap year mode of the arrangement shown above has another surprise in store that takes us ever deeper into understanding how we might be expected to use Kryptos in light of this new knowledge. Remember that the conventional 365 day mode of the sculpture coded for WESTIDBYROWS in the green calendar section of row 14 above. With the additional letter put back the plain text now reads WESTXLAYERTWO. The next clue is something of a creative leap that was found purely by chance, but if any reasoning is to be given in hindsight for this step then it is the initial observation of symmetry between the bottom and top of the sculpture. Imagine then that row 15 represents a reflection of row 14. Look at the correspondence of the letters in the calendar portion of rows 14 and 15.

T	E	S	F	O	R	T	Y	F	O	U	R	S	E	C	O	N	D	S	W	E	S	T	X	L	A	Y	E	R	T	W	O		
S	L	O	W	L	Y	D	E	S	P	A	R	A	T	L	Y	S	L	O	W	L	Y	T	H	E	R	E	M	A	I	N	S		

Let's rearrange the letters in the calendar section to arrive at the best possible match to the row above.

T	E	S	F	O	R	T	Y	F	O	U	R	S	E	C	O	N	D	S	W	E	S	T	X	L	A	Y	E	R	T	W	O		
S	L	O	W	L	Y	D	E	S	P	A	R	A	T	L	Y	S	L	O	W	E	S	T	H	L	A	Y	E	R	M	I	N		

The resulting anagram is WESTHLAYERMIN or to be more explicit WEST H LAYER MIN. The H and Min is obviously an allusion to Hours and Minutes, so with this in mind let's reappraise the coordinates we're given in K2, which we should remember are also the coordinates of the sculpture itself.

Part IV

The Coordinates

The coordinates of the sculpture itself as given in K2 are quite remarkable. The bigger question that will no doubt be asked post this paper will be, which came first, the coordinates or the CIA, by which I mean, is this a coincidence or was the siting of the original building itself chosen on the basis of these coordinates? Either way it should remind us that there is an inherent human quality of constantly seeking out the underlying beauty of mathematical relationships in nature. This appears to have existed in all cultures throughout all history and remains a fundamental aspect of our higher selves that seems to persist even through the darkest ages of ignorance.

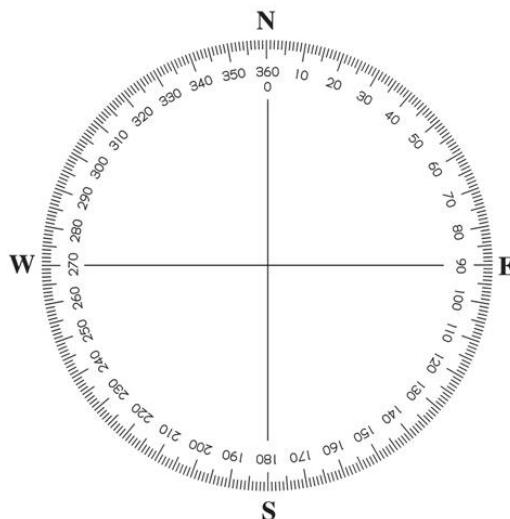
Returning to the discovery made at the end of the last section with regards the relevance of the hours and minutes. If we take the anagram located in row 15 at its word and only concentrate on the hours and minutes of the K2 coordinates then we end up with the following relationship when summing the numbers.

$$38 + 57 + 77 + 8 = 180$$

We can see that hours and minutes were considered equivalent in arriving at this result, so let's try the same method, but now add the seconds as well. We'll treat each individual component as a discrete number even if it is fractional.

$$180 + 6 + 5 + 44 = 235$$

This result is absolutely key as 235 is the number of synodic months in what is known as the Metonic cycle. Once we know that there may be a lunar connection to Kryptos we can start reappraising other aspects of what we already know. Firstly we know that two complete cycles (or traversal of the embedded coordinate system) of the Hours and Minutes will symbolically denote the traversing of 360 degrees.



It's at this point that we reach our first 'jump point' whereby we can make a deductive leap and arrive at a profound result. The result is proof that there is most likely a hardware model in play designed to be an integral part of the decryption process. A quick note before proceeding is that the coordinate traversal requires revisiting rows 1 and 2 after we have reached 38. They are then traversed a second time before the traversal of row 3 giving a count of 57. I point this out to show that there are indeed 360 squares in the coordinate row traversal.

The Leap

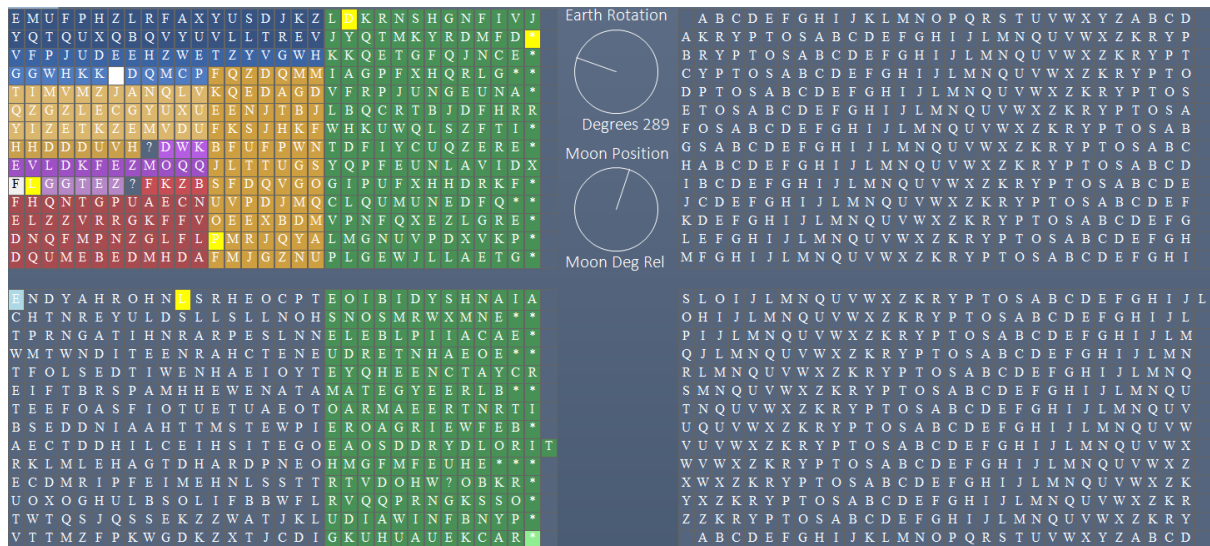
If for every 180 degree pass through the Hours and Minutes coordinate model we accrue an extra 6.5 degrees (as defined symbolically by the last block of 12 characters in the first set of coordinates) then we eventually accrue a total of 13 degrees extra for each 360 traversal. This is significant as 13 degrees represents the apparent motion of the moon through the sky each day. If we use the 28 spaces in the calendar block to record each time that we traverse 13 degrees then we get a pretty accurate method of deriving the 13 lunar months of the year. We know from our calculation of the figure 235 above that we are being asked to be flexible about the change of units from hours and minutes to pure degrees, so it is not a great leap to suppose that the same is true for the seconds. In this case once we have traversed the coordinate blocks for the Hours and Minutes corresponding to 180 squares we can switch to the block of 12 characters that symbolises 6.5 degrees. We can choose to either ignore the ? symbol or not, but if we do ignore it then every time the cycle repeats we will have skipped forward 192 characters.

This skip sequence will be immediately recognisable as the algorithm required to extract K3. All we have to do is set a counter off in K3 that traverses 1 square for every square in the coordinate traversal plus the squares in the lunar adjustment register. At the point the cycle repeats we should record each letter currently under the K3 traversal counter. If the cycle was left to run indefinitely then we would eventually end up at a period of 19 years at which we would have an integer correlation between the number of sidereal moon months and the number of solar years. This relationship shows 254 sidereal moon months for every 19 solar years. There are also 235 synodic months in the same period, which we know from the calculation above we are also capable of measuring using simply the full set of coordinates themselves.

[A note about the Earth's sidereal motion. The 360 degree traversal is assumed to represent the sidereal rotation of the earth and thus represent one sidereal day, which it should be pointed out is 23 hours 56 minutes, not 24 hours. This small 4 minute discrepancy adds up to one day in a whole year due to the fact that one orbit of the sun is equivalent to a day itself. To compensate for the solar day versus the sidereal day we only need to add another square for each 360 degree traversal. This can be done by including the ? symbol from the lunar runoff as part of our row traversal. The effect of this is that we move the equivalent of 24 hours rather than 23 hours 56 minutes, which is the Earth's sidereal period. Employing such a scheme however causes us to move one place back in the coordinate traversal for every 360 degree rotation. For the initial computation shown in the computer model there is no adjustment for sidereal vs solar day.]

This concept is best portrayed programmatically, so a computer model has been made using C# to illustrate this point and is available to copy and reconstruct from the source code in the appendices.

The illustration below shows the calendar cursor moving to day two and the first moon phase cursor moving to the first position of 28 after 13 degrees of lunar traversal.



There is still much more to be said about the set-up of the various registers in the hardware whose uses will become apparent as we progress, but I think there is value in stopping at this point and consolidating our position by gaining additional help from a useful but unexpected outside resource in the guise of the novel Contact by Carl Sagan.

Part V

The Contact Connection

It was mentioned in the introduction that I had chanced upon a paperback copy of Carl Sagan's novel *Contact* in 2013 after creating the last paper in this series. Key chapters and the unfolding of the story itself seem to parallel in some detail what has already been laid out so far in this paper. I will outline the section and chapter headings here for future reference.

Part 1. The Message

1. Transcendental Numbers
2. Coherent Light
3. White Noise
4. Prime Numbers
5. Decryption Algorithm
6. Palimpsest
7. The Ethanol in W-3
8. Random Access
9. The Numinous

Part 2. The Machine

10. Precession of the Equinoxes
11. The World Message Consortium
12. The One-Delta Isomer
13. Babylon
14. Harmonic Oscillator
15. Erbium Dowel
16. The Elders of Ozone
17. The Dream of the Ants
18. Superunification

Part 3. The Galaxy

19. Naked Singularity
20. Grand Central Station
21. Causality
22. Gilgamesh
23. Reprogramming
24. The Artist's Signature

To recap, the big break in finding the steganographic scheme in play came down to finding those initial repeating blocks that were 19 letters long. 19 of course is a prime number, so let's see how many other chapters we have a possible connection to.

3. White Noise. The initial Kryptos text 'Message' appears to be a random collection of letters.

6. Palimpsest. The keyword for decrypting K1 and also an indication that we may have to look at the sculpture in more detail in order to find more information hidden underneath.

2. Coherent Light. By solving the cryptic K1 clue through the removal of the word 'light' and correlating the cipher and plain text we uncover the underlying steganographic scheme used to hide the coordinates and the calendar.

4. Prime Numbers. Our initial break was the discovery of the repeating blocks of 19 letters as mentioned above.

1. Transcendental Numbers. We've found approximations to both π and e (both transcendental numbers) in the number relationships beyond the core coordinates.

24. The Artist's Signature. (See appendices).

5. Decryption Algorithm. It's in this chapter that we learn the repeating prime numbers were only the tip of the iceberg and that further information had been hidden using polarisation and phase modulation of the original radio signal. The use of polarisation seems to link directly to the observations that the ordering of the data may well be a factor if we are to follow the clues relating to the North and South anagrams identified earlier. For the importance of phase we will revisit the findings of the last paper in this series that seem to imply the position of the cursors when traversing the coordinate system are relevant in determining which letters in K4 we should be focused on. We already know for instance that UNYY codes for OBKR to give KRYPT and that this is an output of the traversal of the first coordinate block.

The biggest hint that we need to take from the story irrespective of all the other evidence is that the message once decrypted contains instructions for how to build a machine. We have already seen at this early stage that we are being pointed in the direction of a mechanism that first has to be constructed before we can run our decryption algorithms correctly. Such a mechanism is not without precedence and we find the perfect analogue in the guise of a lesser known archaeological artefact called the Antikythera Mechanism. This will be discussed in more detail in the next section, but in short it is an example of one of the first ever analogue computers.

Continuing along this line of reasoning that we are somehow expected to construct something we are faced with another clue in Contact at the beginning of 'Part 2. The Machine'. On entering this section of the novel we are greeted by a quote from Thomas Paine's 'Age Of Reason' which reads.

*'The Almighty Lecturer, by displaying the principles of science in the structure of the universe, has invited man to study and to imitation. It is as if He has said to the inhabitants of this globe that we call ours, "**I have made an earth for man to dwell upon, and I have rendered the starry heavens visible, to teach him science and the arts.** He can now provide for his own comfort, and learn from my munificence to all to be kind to each other.'*

With these clues in mind let's take a more in depth look at the Antikythera Mechanism.

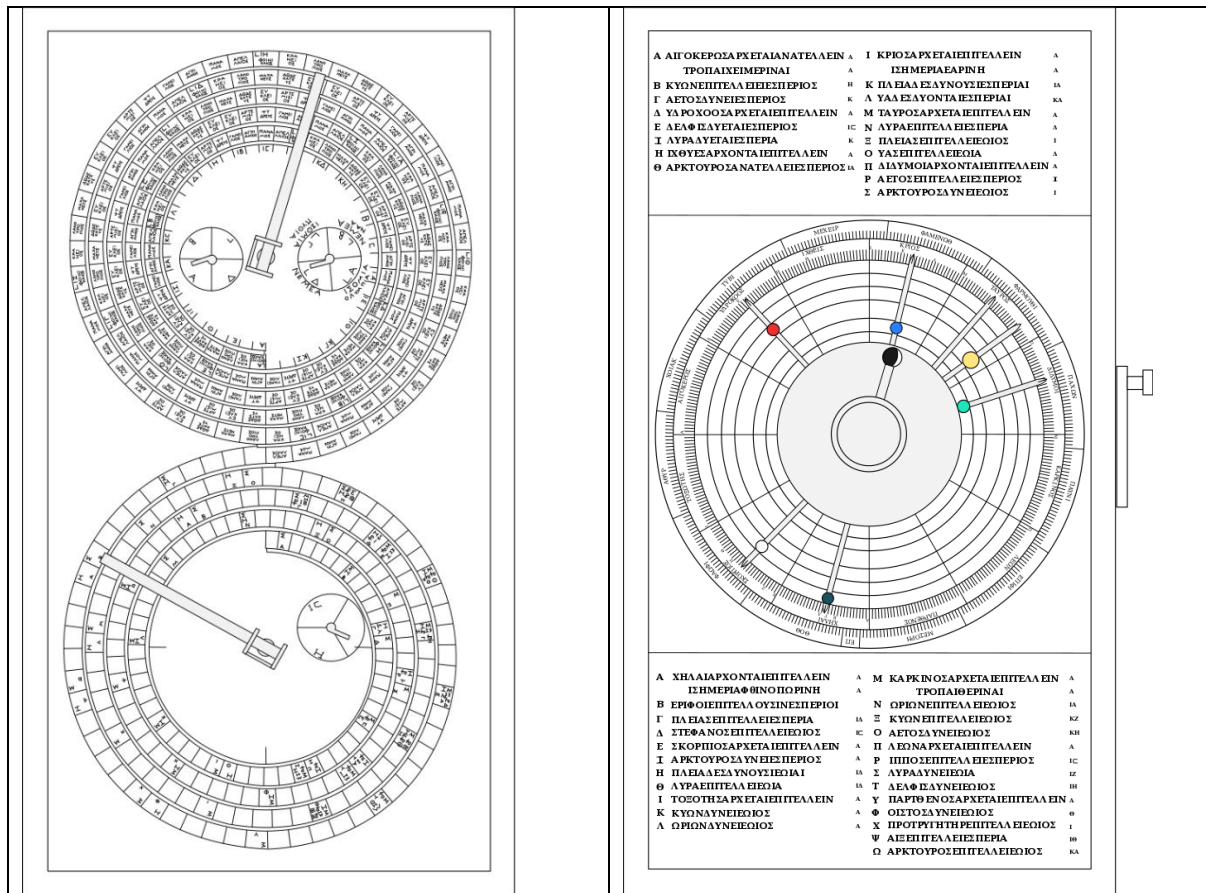
Part VI

The Antikythera Mechanism

For those not familiar with the Antikythera Mechanism it is essentially the remains of some heavily corroded metal gears found on a ship wreck off the island of Antikythera at the very beginning of the 20th century. This strange artefact lay unclassified in a Greek museum for some time before archaeologists suddenly noted that the gearing ratios in the mechanism appeared to relate to celestial cycles. As the sophistication of the device became apparent people started to refer to the mechanism as an object seemingly displaced in time as it encapsulated a detailed knowledge of celestial mechanics thought impossible to have been known at the time of construction. In short the mechanism encapsulates the workings of both the Metonic and Saros cycles of 19 and approximately 18 years respectively. These cycles are important as they can be used to predict the days on which solar and lunar eclipses are likely to occur. The aspects of eclipse prediction will be revisited in a later section, but first lets acquaint ourselves with a reconstruction of what the Antikythera Mechanism looked like. A good starting point as ever is the Wikipedia entry at: https://en.wikipedia.org/wiki/Antikythera_mechanism, but we should at least note here that there were three major dials that could be read off from the mechanism. The mechanism had a front and back face, with the front having a single dial and the back having two dials. There was a winding arm in the mechanism that would allow the operator to animate the device and show the evolution of the relative positions of the moon and planets with respect to time. The largest remaining part of the mechanism recovered from the seabed can be seen below. It was only with the advent of advanced imaging techniques that the true complexity of the gearing became apparent.



Once reconstructed, the front and back dials of the mechanism looked as follows. The dials of most interest to us are the ones on the back face of the device as shown here on the left as they represent both the Metonic and Saros cycles respectively.



For anyone looking to find out more about the Antikythera mechanism after reading this paper then an excellent resource is the recent book on the subject 'A portable Cosmos' by Alexander Jones, which is a very up to date summation of all the history , research and recent conclusions. One of the most remarkable aspects of the mechanism was that it was covered in something akin to an instruction manual that gave a good indication of how it should be used. It's conjectured in the following chapters that the recovery of K4 plaintext is simply piecemeal recovery of such instructions and that a further puzzle awaits once we know how to operate Kryptos properly.

Part VII

Dial Instructions?

An interesting aspect of some of the K4 decryption attempts recapped in this paper are that words occasionally seem to jump out in isolation independently of a complete solution, with a classic case being the string 'KRYPTO' described in the earlier chapters. In 2015 I found a speculative method for decrypting BERLIN and CLOCK in the correct positions (64-74) along with the words BINARY and KNOW starting in position 44 using the same method outlined in the previous paper. It should be pointed out up front that this is an algorithm that uses masking and substitution, which is a completely different method to the popular technique circulating in the internet forums about a skip sequence of 38 following an offset of 77.

That technique may well prove in time to have merits especially in light of the hardware model described in this paper as it feels like a much more natural fit than the technique I'm about to describe. I shall detail the method shortly, but my gut feel is that these titbits are merely instructional pointers or more clues rather than the real thing. Combining the evidence I would guess that K4 is made up of multiple parts and is designed to help clarify the hardware set-up and algorithm that we need to run on the machine in order to extract the final message, but this may not necessarily relate to the final message at all as an algorithm would be a liberty to extract letters from anywhere in the sculpture on a cyclical basis. The additional idea developed here that any plaintext once decrypted may be relevant to the location of its find also fits well with the analogue of the Antikythera mechanism. As mentioned in the previous section the machine was also inscribed with many symbols that outlined its function. In the novel Contact there is again a similar allusion to this concept in the guise of a hidden primer that details how to fully construct 'The Machine' once discovered.

Before revisiting the relevance of what is believed to be discoverable plaintext I'll outline the algorithm in detail so that others can replicate the work and either disprove its relevance or perhaps use it to find more discernible plaintext if they can find the correct 'Row Masks'.

Row Masks

It should be pointed out up front that this section has a great deal more conjecture and creative license than any of the previous sections, so I had initially thought about adding the results obtained as an entry in the appendix at the back of the paper. Its inclusion here may at first seem jarring, but the result achieved appears to be relevant to the phenomenon we're dealing with, so it was thought best to present the evidence first before making the next steps with what is thought to be the recovered K4 plaintext for positions 44 - 74.

An interesting phenomena that was highlighted in the last paper was the symmetrical occurrence of anagrammed words in mirrored rows. Just as we have previously seen NORTH, SOUTH, EAST and WEST anagrammed with missing letters in the vertical plane it was noticed that these relationships also existed in the horizontal plane. The relationship put forward in that last paper is recapped here. DFROM had the letters in the row re-arranged in the last paper in order to form the word FIELD and SGATH became SOFTH again using re-arrangement of letters within the row. The scheme that follows keeps the letters in their original positions as shown below. The orange highlight shows the letters that will end up being masked from the row.

S	H	A	D	I	N	G	A	N	D	T	H	E	A	B	S	E	N	C	B	E	T	W	E	E	N	S	U	B	T	L	E	
E	O	F	L	I	G	H	T	L	I	E	S	T	H	E	N	U	A	N	C	E	O	F	I	Q	L	U	S	I	O	N	*	
I	T	W	A	S	T	O	T	A	L	L	Y	I	N	V	I	S	I	B	L	E	H	O	W	S	T	H	A	T	P	O	*	
S	S	I	B	L	E	?	T	H	E	Y	U	S	E	D	T	H	E	E	A	R	T	H	S	M	A	G	N	E	T		*	
I	C	F	I	E	L	D	X	T	H	E	I	N	F	O	R	M	A	T	I	O	N	W	A	S	G	A	T	H	E	R	*	
E	D	A	N	D	T	R	A	N	S	M	I	T	T	E	D	U	N	D	E	R	G	R	U	U	N	D	T	O	A	N	U	
N	K	N	O	W	N	L	O	C	A	T	I	O	N	X	D	O	E	S	L	A	N	G	L	E	Y	K	N	O	W	A	*	
B	O	U	T	T	H	I	S	?	T	H	E	Y	S	H	O	U	L	D	I	T	S	B	U	R	I	E	D	O	U	T	*	
T	H	E	R	E	S	O	M	E	W	H	E	R	E	X	W	H	O	K	N	O	W	S	T	H	E	E	X	A	C	T	L	
O	C	A	T	I	O	N	?	O	N	L	Y	W	I	T	H	I	S	W	A	S	H	I	S	L	A	S	T	M	E	S	*	
S	A	G	E	X	T	H	I	R	T	Y	E	I	G	H	T	D	E	G	R	E	E	S	F	I	F	T	Y	S	E	*	*	
V	E	N	M	I	N	U	T	E	S	S	I	X	P	O	I	N	T	F	I	V	E	S	E	C	O	N	D	S	N	O	*	
R	T	H	S	E	V	E	N	T	Y	S	E	V	E	N	D	E	G	R	E	E	S	E	I	G	H	T	M	I	N	U	*	
T	E	S	F	O	R	T	Y	F	O	U	R	S	E	C	O	N	D	S	W	E	S	T	X	L	A	Y	E	R	T	W	O	*
S	L	O	W	L	Y	D	E	S	P	A	R	A	T	L	Y	S	L	O	W	L	Y	T	H	E	R	E	M	A	I	N	S	
O	F	P	A	S	S	A	G	E	D	E	B	R	I	S	T	H	A	T	E	N	C	U	M	B	E	R	E	D	T	*	*	
H	E	L	O	W	E	R	P	A	R	T	O	F	T	H	E	D	O	O	R	W	A	Y	W	A	S	R	E	M	O	V	*	
E	D	W	I	T	H	T	R	E	M	B	L	I	N	G	H	A	N	D	S	I	M	A	D	E	A	T	I	N	Y	*	*	
B	R	E	A	C	H	I	N	T	H	E	U	P	P	E	R	L	E	F	T	H	A	N	D	C	O	R	N	E	R	A	N	
D	T	H	E	N	W	I	D	E	N	I	N	G	T	H	E	H	O	L	E	A	L	I	T	T	L	E	I	I	N	*	*	
S	E	R	T	E	D	T	H	E	C	A	N	D	L	E	A	N	D	P	E	E	R	E	D	I	N	T	H	E	H	O	T	
A	I	R	E	S	C	A	P	I	N	G	F	R	O	M	T	H	E	C	H	A	M	B	E	R	C	A	U	S	E	D	*	
T	H	E	F	L	A	M	E	T	O	F	L	I	C	K	E	R	B	U	T	P	R	E	S	E	N	T	L	Y	D	E	T	A
I	L	S	O	F	T	H	E	R	O	O	M	W	I	T	H	I	N	E	M	E	R	G	E	D	F	R	O	M	*	*	*	
T	H	E	M	I	S	T	X	C	A	N	Y	O	U	S	E	E	A	N	Y	T	H	I	N	G	Q	?	X	X	X	X	*	
X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	*		
X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	B	E	R	*	
L	I	N	C	L	O	C	K	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	*		

It was only some time after the last paper was released that I realised I'd missed a crucial observation in the above scheme. The rows are indeed symmetrical, but the incomplete

anagramming scheme is more subtle than was first suspected. Let's remind ourselves again briefly of one of the early results we had when moving the first block of 13 letters wholesale. You will remember that we saw a very distinct pattern emerge.

S	H	A	D	I	N	G	A	N	D	T	H	E	A	B	S	E	N	C	B	E	T	W	E	E	N	S	U	B	T	L	E		
E	O	F	L	I	G	H	T	L	I	E	S	T	H	E	N	U	A	N	C	E	O	F	I	Q	L	U	S	I	O	N			
I	T	W	A	S	T	O	T	A	L	L	Y	I	N	V	I	S	I	B	L	E	H	O	W	S	T	H	A	T	P	O			

If we examine the original symmetry relationship again with this in mind then we find an interesting correlation.

S	H	A	D	I	N	G	A	N	D	T	H	E	A	B	S	E	N	C	B	E	T	W	E	E	N	S	U	B	T	L	E	
E	O	F	L	I	G	H	T	L	I	E	S	T	H	E	N	U	A	N	C	E	O	F	I	Q	L	U	S	I	O	N	*	
I	T	W	A	S	T	O	T	A	L	L	Y	I	N	V	I	S	I	B	L	E	H	O	W	S	T	H	A	T	P	O	*	
S	S	I	B	L	E	?	T	H	E	Y	U	S	E	D	T	H	E	E	A	R	T	H	S	M	A	G	N	E	T		*	
I	C	F	I	E	L	D	X	T	H	E	I	N	F	O	R	M	A	T	I	O	N	W	A	S	G	A	T	H	E	R	*	
E	D	A	N	D	T	R	A	N	S	M	I	T	T	E	D	U	N	D	E	R	G	R	U	U	N	D	T	O	A	N	U	
N	K	N	O	W	N	L	O	C	A	T	I	O	N	X	D	O	E	S	L	A	N	G	L	E	Y	K	N	O	W	A	*	
B	O	U	T	T	H	I	S	?	T	H	E	Y	S	H	O	U	L	D	I	T	S	B	U	R	I	E	D	O	U	T	*	
T	H	E	R	E	S	O	M	E	W	H	E	R	E	X	W	H	O	K	N	O	W	S	T	H	E	E	X	A	C	T	L	
O	C	A	T	I	O	N	?	O	N	L	Y	W	W	T	H	I	S	W	A	S	H	I	S	L	A	S	T	M	E	S	*	
S	A	G	E	X	T	H	I	R	T	Y	E	I	G	H	T	D	E	G	R	E	E	S	F	I	F	T	Y	S	E	*	*	
V	E	N	M	I	N	U	T	E	S	S	I	X	P	O	I	N	T	F	I	V	E	S	E	C	O	N	D	S	N	O	*	
R	T	H	S	E	V	E	N	T	Y	S	E	V	E	N	D	E	G	R	E	E	S	E	I	G	H	T	M	I	N	U	*	
T	E	S	F	O	R	T	Y	F	O	U	R	S	E	C	O	N	D	S	W	E	S	T	X	L	A	Y	E	R	T	W	O	*
S	L	O	W	L	Y	D	E	S	P	A	R	A	T	L	Y	S	L	O	W	L	Y	T	H	E	R	E	M	A	I	N	S	
O	F	P	A	S	S	A	G	E	D	E	B	R	I	S	T	H	A	T	E	N	C	U	M	B	E	R	E	D	T	*	*	
H	E	L	O	W	E	R	P	A	R	T	O	F	T	H	E	D	O	O	R	W	A	Y	W	A	S	R	E	M	O	V	*	
E	D	W	I	T	H	T	R	E	M	B	L	I	N	G	H	A	N	D	S	I	M	A	D	E	A	T	I	N	Y	*	*	
B	R	E	A	C	H	I	N	T	H	E	U	P	P	E	R	L	E	F	T	H	A	N	D	C	O	R	N	E	R	A	N	
D	T	H	E	N	W	I	D	E	N	I	N	G	T	H	E	H	O	L	E	A	L	I	T	T	L	E	I	I	N	*	*	
S	E	R	T	E	D	T	H	E	C	A	N	D	L	E	A	N	D	P	E	E	R	E	D	I	N	T	H	E	H	O	T	
A	I	R	E	S	C	A	P	I	N	G	F	R	O	M	T	H	E	C	H	A	M	B	E	R	C	A	U	S	E	D	*	
T	H	E	F	L	A	M	E	T	O	F	L	I	C	K	E	R	B	U	T	P	R	E	S	E	N	T	L	Y	D	E	T	A
I	L	S	O	F	T	H	E	R	O	O	M	W	I	T	H	I	N	E	M	E	R	G	E	D	F	R	O	M	*	*	*	
T	H	E	M	I	S	T	X	C	A	N	Y	O	U	S	E	E	A	N	Y	T	H	I	N	G	Q	?	X	X	X	X	*	
X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	*	
X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	B	E	R	*	
L	I	N	C	L	O	C	K	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	*	

The relationship in the bottom right does not seem to fit the pattern although there is an anagram pattern of EARTH in the vertical plane that we'll see later maybe of relevance. The relationship of real interest is the interplay between the words FIELD and SOUTH.

The previous paper outlines the reasoning behind why the top row is of interest due to its proximity to the row in the coordinate block that specifies positions 64 – 70. Just to re-cap that point I'll explain here again that if we take the coordinate block for 77 and look at the individual rows then we end up with the following implied numbering scheme.

S	E	D	T	H	E	E
N	F	O	R	M	A	T
T	T	E	D	U	N	D
O	N	X	D	O	E	S
Y	S	H	O	U	L	D
R	E	X	W	H	O	K
W	W	T	H	I	S	W
I	G	H	T	D	E	G
X	P	O	I	N	T	F
V	E	N	D	E	G	R
S	E	C	O	N	D	S

77	76	75	74	73	72	71
64	65	66	67	68	69	70
63	62	61	60	59	58	57
50	51	52	53	54	55	56
49	48	47	46	45	44	43
36	37	38	39	40	41	42
35	34	33	32	31	30	29
22	23	24	25	26	27	28
21	20	19	18	17	16	15
8	9	10	11	12	13	14
7	6	5	4	3	2	1

As we had already been given the crib BERLIN for positions 64-69 at that point in time the corresponding row in which this occurred was of interest.

I	C	F	I	E	L	D	X	T	H	E	I	N	F	O	R	M	A	T	I	O	N	W	A	S	G	A	T	H	E	R	*
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

The last paper described how ICXUEI **encrypted** using the Vigenere table and positions 63-69 of K4 yielded the BERLIN result, but the scheme for the substitution of TH for U was pure conjecture. The discovery of the interlocking anagram in the vertical plane suddenly gave another possibility for the substitution scheme in use. There is an enigmatic clue that signs us off at the end of K3 that reads 'CAN YOU SEE ANYTHING Q?'.

There exists a relatively well known encoding scheme called the four square cipher. The scheme uses four 5 X 5 grids each containing all the letters of the alphabet, but with the notable omission of the letter Q! Two of the 5 X 5 grids are primed with keywords which have the effect of reordering the letter position in the grid. The method I stumbled across took the implied relationship between the words FIELD and SOUTH and used these to create a substitution matrix based roughly on the principles of the four square cipher. The resulting scheme can be shown as follows.

F	I	E	L	D		A	B	C	D	E
A	B	C	G	H		F	G	H	I	J
J	K	M	N	O		K	L	M	N	O
P	R	S	T	U		P	R	S	T	U
V	W	X	Y	Z		V	W	X	Y	Z
A	B	C	D	E		A	B	C	D	E
F	G	H	I	J		F	G	H	I	J
K	L	M	N	O		L	M	N	P	R
P	R	S	T	U		V	W	X	Y	Z
V	W	X	Y	Z		S	O	U	T	H

Using this arrangement as the basis for our substitution matrix we can see that a suitable substitution for TH would be UY, which would not fit our requirements in terms of TH being replaced by U.

F	I	E	L	D		A	B	C	D	E
A	B	C	G	H		F	G	H	I	J
J	K	M	N	O		K	L	M	N	O
P	R	S	T	U		P	R	S	T	U
V	W	X	Y	Z		V	W	X	Y	Z
A	B	C	D	E		A	B	C	D	E
F	G	H	I	J		F	G	H	I	J
K	L	M	N	O		L	M	N	P	R
P	R	S	T	U		V	W	X	Y	Z
V	W	X	Y	Z		S	O	U	T	H

If we play around with different schemes then we are left with a better substitution for TH which would fit this scheme, in this case being a digraph of TU or UU. Under such a scheme we're still left with a 2 for 2 substitution, but the conjectured aspect of self-nullification leaves us with the required single U. This may at first seem like too much conjecture, but as more examples are added the concept does *appear* to accrue some meaningful results.

If we play with rotations of the individual matrices themselves and perform anti clockwise rotations of the matrices in the top left and bottom left and top left before applying a 180 rotation to the matrix in the bottom left then we achieve the cleaner result that we were looking for.

Z	Y	X	W	V		A	B	C	D	E
U	T	S	R	P		F	G	H	I	J
O	N	M	K	J		K	L	M	N	O
H	G	C	B	A		P	R	S	T	U
D	L	E	I	F		V	W	X	Y	Z
Z	Y	X	W	V		E	K	R	Z	H
U	T	S	R	P		D	J	P	Y	T
O	N	M	L	K		C	I	N	X	U
J	I	H	G	F		B	G	M	W	O
E	D	C	B	A		A	F	L	V	S

Post 2013 we are in an exciting position as we have a crib for the word CLOCK in positions 70 - 74 and position 70 is still within our focus for this row. As a result we have an idea as to what any relevant substitution might be.

If we first work backwards and find what the Vigenere encryption of CLOCK would be for positions 70 – 74 for K4 then we arrive at the following.

PT	CLOCK
KW	MZFPK
CT	CBAQK

The first thing to be aware of when viewing this seemingly meaningless result is that I was already hard wired to be looking for another palimpsest in that particular row that would be in line with the method presented in the previous paper, so from that perspective it was easier for me to spot the following relationships rather than coming in cold. The point of interest I had noticed was that the cipher text in the next distinct block of 7 characters for that row had the following letters.

T I M V M Z J A N Q L V K Q E D A G D V F R P J U N G E U N A *

Reversing this block of 7 from the 77 coordinate array gives DGADEQK. Looking again at the plaintext for the entire row we can now make an observation that wasn't apparent before. Amongst the plaintext we can see that there are multiple digraphs of letters corresponding to the cardinal points of the compass, i.e. NORTH, SOUTH, EAST and WEST.

Let's examine the complete plaintext of this row again with this in mind.

I C F I E L D X T H E I N F O R M A T I O N W A S G A T H E R *

The digraphs of letters contained within the cardinal points are highlighted in red and are TH, OR, AT and ON respectively. What is so intriguing about this result is the positioning of the digraphs in the block of 7 characters that are currently of interest with regards CLOCK.

If we superimpose the digraphs on top of the cipher text then we see the following:

T I M V M Z J A N Q L V K Q E D A G D V F R P J U N G E U N A *

That would seem to imply that a substitution that nullified ED completely and changed GD to BC would in fact give us a string that when parsed backwards would give us the result we're looking for. Can we achieve such a result using a similar method to the one used for the BERLIN substitution?

Let's initially try the same four square matrix as we ventured before.

F	I	E	L	D		A	B	C	D	E
A	B	C	G	H		F	G	H	I	J
J	K	M	N	O		K	L	M	N	O
P	R	S	T	U		P	R	S	T	U
V	W	X	Y	Z		V	W	X	Y	Z
A	B	C	D	E		A	B	C	D	E
F	G	H	I	J		F	G	H	I	J
K	L	M	N	O		L	M	N	P	R
P	R	S	T	U		V	W	X	Y	Z
V	W	X	Y	Z		S	O	U	T	H

Again we can do rotations to find a better scheme. For this one let's mirror the top left matrix and do a left rotation on the bottom left matrix.

D	L	E	I	F		A	B	C	D	E
H	G	C	B	A		F	G	H	I	J
O	N	M	K	J		K	L	M	N	O
U	T	S	R	P		P	R	S	T	U
Z	Y	X	W	V		V	W	X	Y	Z
A	B	C	D	E		E	K	R	Z	H
F	G	H	I	J		D	J	P	Y	T
K	L	M	N	O		C	I	N	X	U
P	R	S	T	U		B	G	M	W	O
V	W	X	Y	Z		A	F	L	V	S

Using the exact same technique as before we are given NO, but if we expand our general rule from before to say that no substitution of a cardinal point can transform into any letters of that same cardinal point then we are left with the complete nullification of OR in the underlying plaintext. This in itself is very interesting as the nullification would have to take place after encryption has taken place. Do we have an example of this? There has already been a lot of discussion about the intentional omission of the X in the K2 plaintext, but now knowing what we know about the calendar scheme we can surmise that this was intentional and that the 'MIST X' in K3 is in itself a cryptic clue, a theory already well established by others with good evidence. What really interests us with regards this scheme is the fact that ESW was changed to EW in the final cut. If the implications of that omission are not immediately obvious then I'll point out explicitly that **the digraphs occurring in the cardinal points are somehow relevant to the decryption scheme**. My overall technique using the foursquare cipher may be some way off what was intended, but my gut feel is that the use of digraph substitution will eventually hold in whatever scheme is found to be correct.

With this in mind let's play around with the four square cipher some more to see if we can cajole the GD into being the required BC. After trying some different permutations of letter orders for the cipher the following scheme was arrived at. The FIELD matrix in the top left has been mirrored in the X and Y plane and the alphabet matrix in the bottom left has been mirrored purely in the X plane. SOUTH is now the first row of the bottom right hand matrix. In this scheme the classic four square relationship for substitution is noted as GD is seen to encode directly to YP.

Z	Y	X	W	V		A	B	C	D	E
U	T	S	R	P		F	G	H	I	J
O	N	M	K	J		K	L	M	N	O
H	G	C	B	A		P	R	S	T	U
D	L	E	I	F		V	W	X	Y	Z
V	W	X	Y	Z		S	O	U	T	H
P	R	S	T	U		A	B	C	D	E
K	L	M	N	O		F	G	I	J	J
F	G	H	I	J		L	M	N	P	R
A	B	C	D	E		V	W	X	Y	Z

If we substitute NFORMAT with NFORMYP and encrypt again using the keyword ABSCISSA then we arrive at a string of KQEDABC. We know that we are already going to nullify the underling OR, so the final string we are left with is KQABC.

As mentioned before I freely admit that in the entire paper that this is the most tenuous of all the proofs offered as there seems to be a lack of clarity as to the rules of when:

To move backwards or forwards through a string.

When to do a replacement in the plaintext based on the corresponding cipher text.

When and how to manipulate the four square matrix itself in order to arrive at the correct substitution result. It may be that there are some hard and fast rules that allow this technique to be extended further and that the conclusions above will eventually make sense as being a subset of examples from a wider scheme. For the purposes of this paper though I am purely going to take the technique already discussed and apply it to the mirror row of the 'Row Masking' scheme. As we saw earlier it is conjectured that the row masking is mirrored, which gives us a top and bottom row for the FIELD SOUTH scheme. If we now look at the bottom (or reflected) row and compare the cipher and plain text then we see the following.

I	L	S	O	F	T	H	E	R	O	O	M	W	I	T	H	I	N	E	M	E	R	G	E	D	F	R	O	M	*	*	*
R	K	L	M	L	E	H	A	G	T	D	H	A	R	D	P	N	E	O	H	M	G	F	M	F	E	U	H	E	*	*	*

Once we have applied the masking (SOFTH and DFROM) let's examine the strings that we have left for digraphs of cardinal points.

I L S O F T H E R O O M W I T H I N E M E R G E D F R O M * * *

R K L M L E H A G T D H A R D P N E O H M G F M F E U H E * * *

We can see that we're now left in a similar position to the issue faced with X LAYER TWO in that we have three letters in proximity that can all participate in the cardinal point NORTH. If we observe the K2 example and drop the middle O then we are left with RO.

A substitution of NO would lead to RO becoming N, but there is at least one other possibility presented here which allows for a substitution of TU, which is a mirroring of the bottom left in the Y plane and a 90 degree left rotation of the bottom right hand matrix.

F	I	E	L	D		A	B	C	D	E
A	B	C	G	H		F	G	H	I	J
J	K	M	N	O		K	L	M	N	O
P	R	S	T	U		P	R	S	T	U
V	W	X	Y	Z		V	W	X	Y	Z
E	D	C	B	A		E	K	R	Z	H
J	I	H	G	F		D	J	P	Y	T
O	N	M	L	K		C	I	N	X	U
U	T	S	R	P		B	G	M	W	O
Z	Y	X	W	V		A	F	L	V	S

Replacing the GTD in the cipher text with TU gives as a final string in the first coordinate block of RKATUH which when reversed reads HUTAKR.

We know from the conjectured row numbering scheme that we are in row 44 as shown below. (Alternatively, using the 77 square coordinate block we would be in row 8 – 14).

S	E	R	T	E	D	T	H	E	C	A	N
A	I	R	E	S	C	A	P	I	N	G	F
T	H	E	F	L	A	M	E	T	O	F	L
I	L	S	O	F	T	H	E	R	O	O	M

1	2	3	4	5	6	7	8				
20	19	18	17	16	15	14	13	12	11	10	9
21	22	23	24	25	26	27	28	29	30	31	32
44	43	42	41	40	39	38	37	36	35	34	33

If we find the K4 string for positions 43 onwards we have

S	S	E	K	Z	Z	W	A	T	J	K	L	U	D	I	A	W	I	N	F	B	N	Y	P
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Decrypting the string HUTAKR with SSEKZZ gives us the word BINARY. This in itself is encouraging, but a scan along the row gives us another interesting result. The encryption of ENIIW against WATJK gives us BKNOW and finally MFGMH against WINFB gives IRSTV. These results seem random at first glance, but they do conform to both the masking and digraph substitution rule. There is still the confusion over when to encrypt and when to decrypt and when to run forward rather than backward, but all the same there does appear to be a phenomenon here that either needs to be embraced or explained away.

The most intriguing aspect of this result is that we may finally be being given our first glimpse of something beyond K4. I say this as there is a gap of 5 letters in the sets of strings that we have just extracted. However, if we take the plaintext from the 77 coordinate block for the 63 – 70 row then we have NFOR. If we look at the mirror row then the fifth letter is an O. Putting all these string together once the O has been treated as being appended to the MFGMH string gives us: BINARY B KNOWN FOR FIRST V BERLIN CLOCK. This string if correct is intriguing because there is a reference to the first 5 of something and we now know that if this string is correct that there is a block of at least 5 letters that doesn't code for anything in this scheme, which is the 5 letter string LUDIA in K4.

The decryption tableau on the right hand side of the sculpture can also be set in a grid system that has 364 days in the top 14 rows and 365 in the bottom 14 rows due to the extraneous L at the end of the 15th row. Such a scheme however does leave 5 letters at the beginning of each row spare. Whether or not this has anything to do with the above is conjecture, but worthy of note. If the first five is something in relation to the set theory clock in Berlin then this would be interesting as the first five minutes are represented by one of the lights in the row of 11 as will be explained in a later section.

Further exploration of the plaintext gives the possibility of yet more rows that share similar properties such as the row below that uses row masks of EARTH and LIGHT. The reader will

notice the exact same pattern as we saw before with an imperfect FIELD anagram crossing FIELD and SOUTH crossing SOUTH, before a third arrangement showing FIELD and SOUTH crossed together

S	H	A	D	I	N	G	A	N	D	T	H	E	A	B	S	E	N	C	B	E	T	W	E	E	N	S	U	B	T	L	E	
E	O	F	L	I	G	H	T	L	I	E	S	T	H	E	N	U	A	N	C	E	O	F	I	Q	L	U	S	I	O	N	*	
I	T	W	A	S	T	O	T	A	L	L	Y	I	N	V	I	S	I	B	L	E	H	O	W	S	T	H	A	T	P	O	*	
S	S	I	B	L	E	?	T	H	E	Y	U	S	E	D	T	H	E	E	A	R	T	H	S	M	A	G	N	E	T		*	
I	C	F	I	E	L	D	X	T	H	E	I	N	F	O	R	M	A	T	I	O	N	W	A	S	G	A	T	H	E	R	*	
E	D	A	N	D	T	R	A	N	S	M	I	T	T	E	D	U	N	D	E	R	G	R	U	U	N	D	T	O	A	N	U	
N	K	N	O	W	N	L	O	C	A	T	I	O	N	X	D	O	E	S	L	A	N	G	L	E	Y	K	N	O	W	A	*	
B	O	U	T	T	H	I	S	?	T	H	E	Y	S	H	O	U	L	D	I	T	S	B	U	R	I	E	D	O	U	T	*	
T	H	E	R	E	S	O	M	E	W	H	E	R	E	X	W	H	O	K	N	O	W	S	T	H	E	E	X	A	C	T	L	
O	C	A	T	I	O	N	?	O	N	L	Y	W	W	T	H	I	S	W	A	S	H	I	S	L	A	S	T	M	E	S	*	
S	A	G	E	X	T	H	I	R	T	Y	E	I	G	H	T	D	E	G	R	E	E	S	F	I	F	T	Y	S	E	*	*	
V	E	N	M	I	N	U	T	E	S	S	I	X	P	O	I	N	T	F	I	V	E	S	E	C	O	N	D	S	N	O	*	
R	T	H	S	E	V	E	N	T	Y	S	E	V	E	N	D	E	G	R	E	E	S	E	I	G	H	T	M	I	N	U	*	
T	E	S	F	O	R	T	Y	F	O	U	R	S	E	C	O	N	D	S	W	E	S	T	X	L	A	Y	E	R	T	W	O	*
S	L	O	W	L	Y	D	E	S	P	A	R	A	T	L	Y	S	L	O	W	L	Y	T	H	E	R	E	M	A	I	N	S	
O	F	P	A	S	S	A	G	E	D	E	B	R	I	S	T	H	A	T	E	N	C	U	M	B	E	R	E	D	T	*	*	
H	E	L	O	W	E	R	P	A	R	T	O	F	T	H	E	D	O	O	R	W	A	Y	W	A	S	R	E	M	O	V	*	
E	D	W	I	T	H	T	R	E	M	B	L	I	N	G	H	A	N	D	S	I	M	A	D	E	A	T	I	N	Y	*	*	
B	R	E	A	C	H	I	N	T	H	E	U	P	P	E	R	L	E	F	T	H	A	N	D	C	O	R	N	E	R	A	N	
D	T	H	E	N	W	I	D	E	N	I	N	G	T	H	E	H	O	L	E	A	L	I	T	T	L	E	I	I	N	*	*	
S	E	R	T	E	D	T	H	E	C	A	N	D	L	E	A	N	D	P	E	E	R	E	D	I	N	T	H	E	H	O	T	
A	I	R	E	S	C	A	P	I	N	G	F	R	O	M	T	H	E	C	H	A	M	B	E	R	C	A	U	S	E	D	*	
T	H	E	F	L	A	M	E	T	O	F	L	I	C	K	E	R	B	U	T	P	R	E	S	E	N	T	L	Y	D	E	T	A
I	L	S	O	F	T	H	E	R	O	O	M	W	I	T	H	I	N	E	M	E	R	G	E	D	F	R	O	M	*	*	*	
T	H	E	M	I	S	T	X	C	A	N	Y	O	U	S	E	E	A	N	Y	T	H	I	N	G	Q	?	X	X	X	X	*	
X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	*		
X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	B	E	R	*
L	I	N	C	L	O	C	K	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	*	

I'm acutely aware that the last section was far too rushed in order to convey the full rationale for the techniques used, so the reader may wish to explore further and experiment using the principles laid out above, but for now we will return to the underlying narrative of the paper which is to show the machine aspects of the underlying number surface and how they relate to the calculation of astronomical time periods.

Part VIII

Putting it all together

Ever since the release of the Clock clue in 2013 there has been speculation with regards the relevance of the Megenlehreuhr Clock in Berlin. If we examine the workings of the clock in more detail then we can see it is broken down as follows. In describing the meaning of the layout we'll proceed in order of time periods.

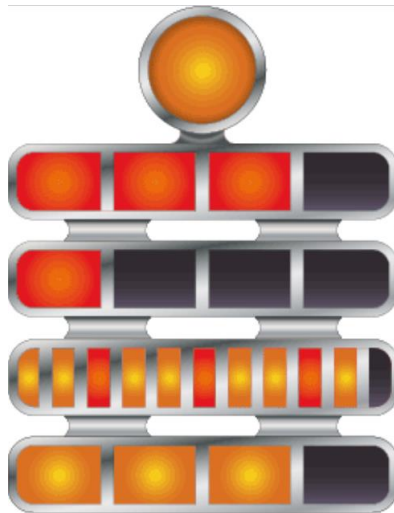
The orange light at the top blinks every second.

The four lights on the bottom row each represent one minute.

The eleven lights in the next row up represent five minute time periods.

The next four lights up represent hours.

The last set of four lights in the top row represent five hours each.



Thinking about the general premise of this clock and that we have the words Berlin Clock in the rows coding for 64- 70, we can make a bold assertion. If a binary scheme is somehow in play and the reference to the set theory clock is relevant then is there a logical place for such a scheme to sit within our mechanism? It is in fact possible to create both a binary clock and a binary calendar using 6 bits and 5 bits respectively. We know that we already have a perfect contender for both a 6 and 5 bit header in the form of the seconds from the latitude coordinates. The presence of 3 non spilt rows beneath them also gives us a good place to try and insert such a scheme. If we are right in our assumption about the mirrored row also making a reference to a binary scheme then let's also look for any indicators in the corresponding mirror rows to see if we have any other additional pointers. First let's take a look at what a binary clock looks like in practice.

Such a scheme is shown here and is lifted directly from the wikipedia entry at:

https://en.wikipedia.org/wiki/Binary_clock

	Hours	Minutes	Seconds
32		1	1
16	0	0	1
8	1	0	0
4	0	1	0
2	1	0	0
1	0	1	1
	10	37	49

If we rotate this scheme 90 anticlockwise and try placing it first in the mirrored row of the plaintext then we see an interesting correlation. The time above will be displayed in binary where the line corresponding to 1, 2, 4, 8, 16, and 32 is represented as THEMIS. You will notice that the end of the row is bounded by the letters HMA. Could this be a direct pointer towards Hours and Minutes. It also seems to be rather coincidental that rearrangement of THEMIS also gives us TIME HS.

S	L	O	W	L	Y	D	E	S	P	A	R	A	T	L	Y	S	L	O	W	L	Y	T	H	E	R	E	M	A	I	N	S	
O	F	P	A	S	S	A	G	E	D	E	B	R	I	S	T	H	A	T	E	N	C	U	M	B	E	R	E	D	T	*	*	
H	E	L	O	W	E	R	P	A	R	T	O	F	T	H	E	D	O	O	R	W	A	Y	W	A	S	R	E	M	O	V	*	
E	D	W	I	T	H	T	R	E	M	B	L	I	N	G	H	A	N	D	S	I	M	A	D	E	A	T	I	N	Y	*	*	
B	R	E	A	C	H	I	N	T	H	E	U	P	P	E	R	L	E	F	T	H	A	N	D	C	O	R	N	E	R	A	N	
D	T	H	E	N	W	I	D	E	N	I	N	G	T	H	E	H	O	L	E	A	L	I	T	T	L	E	I	I	N	*	*	
S	E	R	T	E	D	T	H	E	C	A	N	D	L	E	A	N	D	P	E	E	R	E	D	I	N	T	H	E	H	O	T	
A	I	R	E	S	C	A	P	I	N	G	F	R	O	M	T	H	E	C	H	A	M	B	E	R	C	A	U	S	E	D	*	
T	H	E	F	L	A	M	E	T	O	F	L	I	C	K	E	R	B	U	T	P	R	E	S	E	N	T	L	Y	D	E	T	A
I	L	S	O	F	T	H	E	R	O	O	M	W	I	T	H	I	N	E	M	E	R	G	E	D	F	R	O	M	*	*	*	
T	H	E	M	I	S	T	X	C	A	N	Y	O	U	S	E	E	A	N	Y	T	H	I	N	G	Q	?	X	X	X	X	*	
X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	*		
X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	B	E	R	*
L	I	N	C	L	O	C	K	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	*	

Using five bit binary it is also possible to represent the day of the week, calendar month and day of month in the three line slot adjacent to the binary clock. In choosing which row should perform which count we can take some guidance from the mirrored row above, whereby HMA is replaced by DRL. If we use some creative license to imagine that this might be Days, Rotations and Lunations then we can represent Sunday July 30th as 7, 30 and 7 as follows.

S	H	A	D	I	N	G	A	N	D	T	H	E	A	B	S	E	N	C	B	E	T	W	E	E	N	S	U	B	T	L	E	
E	O	F	L	I	G	H	T	L	I	E	S	T	H	E	N	U	A	N	C	E	O	F	I	Q	L	U	S	I	O	N	*	
I	T	W	A	S	T	O	T	A	L	L	Y	I	N	V	I	S	I	B	L	E	H	O	W	S	T	H	A	T	P	O	*	
S	S	I	B	L	E	?	T	H	E	Y	U	S	E	D	T	H	E	E	A	R	T	H	S	M	A	G	N	E	T		*	
I	C	F	I	E	L	D	X	T	H	E	I	N	F	O	R	M	A	T	I	O	N	W	A	S	G	A	T	H	E	R	*	
E	D	A	N	D	T	R	A	N	S	M	I	T	T	E	D	U	N	D	E	R	G	R	U	U	N	D	T	O	A	N	U	
N	K	N	O	W	N	L	O	C	A	T	I	O	N	X	D	O	E	S	L	A	N	G	L	E	Y	K	N	O	W	A	*	
B	O	U	T	T	H	I	S	?	T	H	E	Y	S	H	O	U	L	D	I	T	S	B	U	R	I	E	D	O	U	T	*	
T	H	E	R	E	S	O	M	E	W	H	E	R	E	X	W	H	O	K	N	O	W	S	T	H	E	E	X	A	C	T	L	
O	C	A	T	I	O	N	?	O	N	L	Y	W	W	T	H	I	S	W	A	S	H	I	S	L	A	S	T	M	E	S	*	
S	A	G	E	X	T	H	I	R	T	Y	E	I	G	H	T	D	E	G	R	E	E	S	F	I	F	T	Y	S	E	*	*	
V	E	N	M	I	N	U	T	E	S	S	I	X	P	O	I	N	T	F	I	V	E	S	E	C	O	N	D	S	N	O	*	
R	T	H	S	E	V	E	N	T	Y	S	E	V	E	N	D	E	G	R	E	E	S	E	I	G	H	T	M	I	N	U	*	
T	E	S	F	O	R	T	Y	F	O	U	R	S	E	C	O	N	D	S	W	E	S	T	X	L	A	Y	E	R	T	W	O	*

So if we have a pulse that ticks on a regular basis then we know that we can relate it to the scheme we found earlier for the solar day and the 360 degree coordinate traversal. The only amendment we have to make is that we will only move one coordinate block for every four minutes of the binary clock.

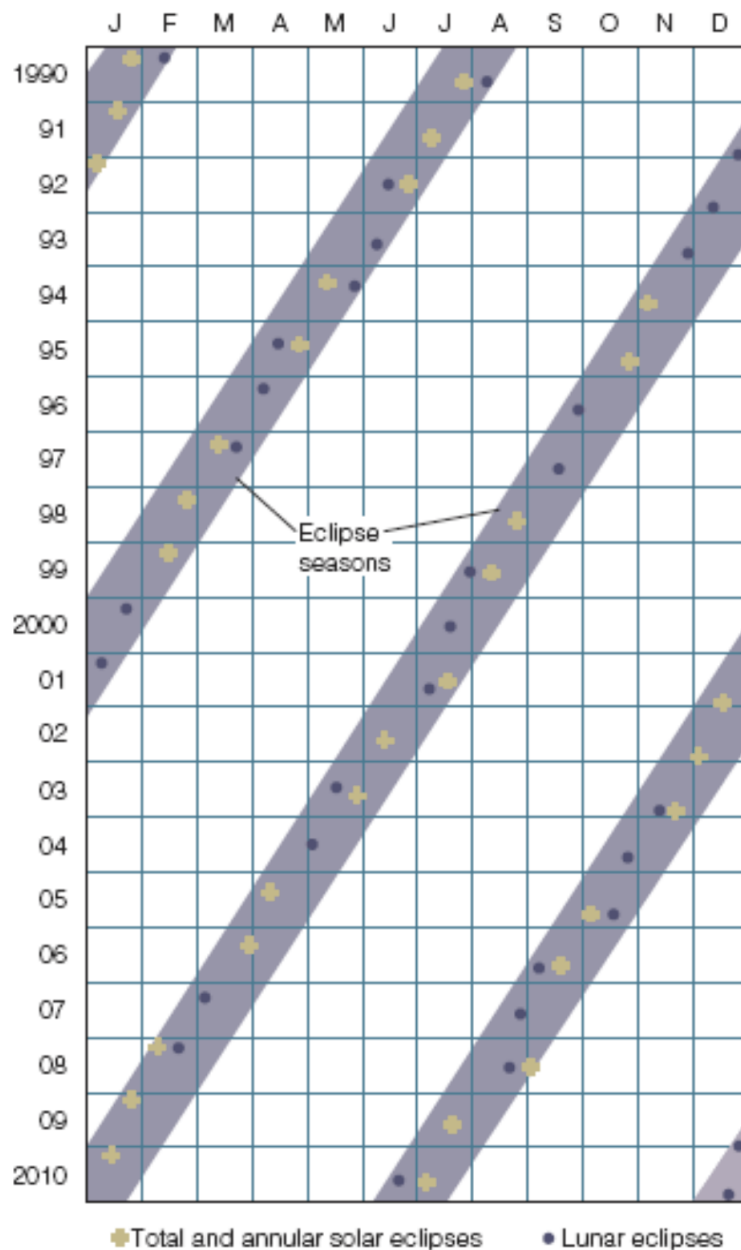
As the intervals of time get larger we can see that we have a means of capturing their evolution and that we can record right up to the value of one Callippic cycle which is equal to 76 years before the sculpture resets itself and starts again. We can guess that the blocks of 52, 7 and 12 relate directly to an analogue of weeks, day of the week and months and are intrinsically bound up with the movement through the Calendar. The spaces shown as asterisks show the evolution of the moon months according to the sidereal motion of the moon, but the synodic months are more tricky to calculate. I will admit that I've fallen short of an elegant way to use the hardware to calculate a synchronised value for the synodic month (without using just the clock itself), but I would surmise that it must be possible as we know that for each synodic month that we can move one block through the hours, minutes and second coordinates. Before starting the cycle again we will have captured 235 synodic months which is almost equal to an integer number of 19 solar years otherwise known as the Metonic cycle.

Just like the set theory clock we saw previously we can see that just before the cycle resets itself at 76 years that we can have a register of 3 rows of 19 years, showing that we are in the final quarter of the Callippic cycle. It is of course possible to show the Callippic progression in years in the 77 coordinate block, but there seems to be a greater elegance to using the first 3 rows of 19. In terms of our start point we already know that there was a full moon the Friday night before November 3rd 1990 so we should be able to sync the machine

to match the correct celestial representations for any time prior and post this date. With regards when to add the leap year we have an unused block of 3 characters above the month recorder of 12 blocks. It may be that this is intended to be a binary counter capable of tracking 8 years at a time and will give us two cues in its cycle for when to add the missing X back into the scheme. Another binary counter of interest can be formed from our block of 8 characters. An 8 bit number is capable of recording numbers up to 255, so this is most likely the register we should be using to capture the passing of our sidereal lunar months.

Eclipse Prediction

Knowing that we have a machine capable of keeping track of synodic months and by extension the Saros cycle, it is very interesting to note that the decryption tableau has a very striking diagonal pattern caused by the effect of each row having a letter offset from the previous. The resulting effect is that the tableau resembles a chart that show eclipse seasons, such as the one shown below.



Part IX

Conclusions

We've seen how the machine once reconstructed works as a timepiece capable of recording time, days, months and years over a 76 year period. What is still to be ascertained is whether it also contains a more subtle method of capturing more complex cycles such as the Saros, which extend over a roughly 18 year period and can be used for eclipse prediction. With the upcoming Eclipse across America on August the 21st 2017 I'm hopeful that there will be an increase in interest from students and other interested parties to use this paper as an initial route into discovering more about how Kryptos may be used as a true analogue computer.

Something that hasn't escaped my attention and that I've spent many walks pondering is another aspect of 'The Machine' that Kryptos forces us to contend with. In 1990 Quantum Cryptography would have been at the very cutting edge of computational and mathematical research, whilst today we take it for granted as a reality of our ever increasing range of technological capabilities. If we think about the way that we have been stepped through the history of cryptography by exposure to so many of the techniques of the past is it not possible that the very concept of Kryptos is a nod towards its future? To be more specific Quantum Mechanics deals with the difficult philosophical questions that arise around the definition of reality and its relationship to measurement. The sculpture itself is inert, but as we've seen our observations of its mathematical properties breathe an imitation of our Solar System into its very being. When we stop looking then it returns to being an inert copper scroll latent with any possible meaning that anyone wishes to ascribe to it. To break Kryptos is to somehow extract information from it and pass information back in to the point where the observed and observer are reflected in some kind of symbiotic computation. Having read around the subject matter of Quantum Mechanics in the past I'm not sure I can find any better analogue of its mysteries than the one presented to us by Kryptos itself.

Part X

Appendices

Appendix 1. Geometrical relationships

The general narrative of the paper revolves around the relationship between Kryptos and the mechanics of keeping solar and lunar time. There appears however to be a nod to a number of geometric relationships that people may want to push further, but have been relegated here to the status of an interesting afterward. It was noted in the previous Valkyrie paper that there appears to be a link between letters in K3 and the initial letters involved in the KRYPTO string described earlier in the paper. The relationship is shown as follows. It should be noted that row 1 is in the modified format where the first 13 letters have been re-arranged.

S	H	A	D	I	N	G	A	N	D	T	H	E	A	B	S	E	N	C	B	E	T	W	E	E	N	S	U	B	T	L	E	
E	O	F	L	I	G	H	T	L	I	E	S	T	H	E	N	U	A	N	C	E	O	F	I	Q	L	U	S	I	O	N	*	
I	T	W	A	S	T	O	T	A	L	L	Y	I	N	V	I	S	I	B	L	E	H	O	W	S	T	H	A	T	P	O	*	
S	S	I	B	L	E	?	T	H	E	Y	U	S	E	D	T	H	E	E	A	R	T	H	S	M	A	G	N	E	T		*	
I	C	F	I	E	L	D	X	T	H	E	I	N	F	O	R	M	A	T	I	O	N	W	A	S	G	A	T	H	E	R	*	
E	D	A	N	D	T	R	A	N	S	M	I	T	T	E	D	U	N	D	E	R	G	R	U	U	N	D	T	O	A	N	U	
N	K	N	O	W	N	L	O	C	A	T	I	O	N	X	D	O	E	S	L	A	N	G	L	E	Y	K	N	O	W	A	*	
B	O	U	T	T	H	I	S	?	T	H	E	Y	S	H	O	U	L	D	I	T	S	B	U	R	I	E	D	O	U	T	*	
T	H	E	R	E	S	O	M	E	W	H	E	R	E	X	W	H	O	K	N	O	W	S	T	H	E	E	X	A	C	T	L	
O	C	A	T	I	O	N	?	O	N	L	Y	W	W	T	H	I	S	W	A	S	H	I	S	L	A	S	T	M	E	S	*	
S	A	G	E	X	T	H	I	R	T	Y	E	I	G	H	T	D	E	G	R	E	E	S	F	I	F	T	Y	S	E	*	*	
V	E	N	M	I	N	U	T	E	S	S	I	X	P	O	I	N	T	F	I	V	E	S	E	C	O	N	D	S	N	O	*	
R	T	H	S	E	V	E	N	T	Y	S	E	V	E	N	D	E	G	R	E	E	S	E	I	G	H	T	M	I	N	U	*	
T	E	S	F	O	R	T	Y	F	O	U	R	S	E	C	O	N	D	S	W	E	S	T	X	L	A	Y	E	R	T	W	O	*
S	L	O	W	L	Y	D	E	S	P	A	R	A	T	L	Y	S	L	O	W	L	Y	T	H	E	R	E	M	A	I	N	S	
O	F	P	A	S	S	A	G	E	D	E	B	R	I	S	T	H	A	T	E	N	C	U	M	B	E	R	E	D	T	*	*	
H	E	L	O	W	E	R	P	A	R	T	O	F	T	H	E	D	O	O	R	W	A	Y	W	A	S	R	E	M	O	V	*	
E	D	W	I	T	H	T	R	E	M	B	L	I	N	G	H	A	N	D	S	I	M	A	D	E	A	T	I	N	Y	*	*	
B	R	E	A	C	H	I	N	T	H	E	U	P	P	E	R	L	E	F	T	H	A	N	D	C	O	R	N	E	R	A	N	
D	T	H	E	N	W	I	D	E	N	I	N	G	T	H	E	H	O	L	E	A	L	I	T	T	L	E	I	I	N	*	*	
S	E	R	T	E	D	T	H	E	C	A	N	D	L	E	A	N	D	P	E	E	R	E	D	I	N	T	H	E	H	O	T	
A	I	R	E	S	C	A	P	I	N	G	F	R	O	M	T	H	E	C	H	A	M	B	E	R	C	A	U	S	E	D	*	
T	H	E	F	L	A	M	E	T	O	F	L	I	C	K	E	R	B	U	T	P	R	E	S	E	N	T	L	Y	D	E	T	A
I	L	S	O	F	T	H	E	R	O	O	M	W	I	T	H	I	N	E	M	E	R	G	E	D	F	R	O	M	*	*	*	
T	H	E	M	I	S	T	X	C	A	N	Y	O	U	S	E	E	A	N	Y	T	H	I	N	G	Q	?	X	X	X	X	*	
X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	*		
X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	B	E	R	*
L	I	N	C	L	O	C	K	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	*	

I raise this as a point of interest as we have seen the K3 decryption points to the 13 degree traversal of the sky by the moon each evening. If K3 has no relation to Tutankhamum then what is this whole diary excerpt about? In light of the above relationship I can't help but wonder if this is yet another play on words with regards the tangent formed in a relationship similar to the above.

A play on words of TU Tan CAR Moon would fit the bill nicely and would be seen to be the angular relationship formed in K4 as follows.

E	C	D	M	R	I	P	F	E	I	M	E	H	N	L	S	S	T	T	R	T	V	D	O	H	W	?	O	B	K	R	*
U	O	X	O	G	H	U	L	B	S	O	L	I	F	B	B	W	F	L	R	V	Q	Q	P	R	N	G	K	S	S	O	*
T	W	T	Q	S	J	Q	S	S	E	K	Z	Z	W	A	T	J	K	L	U	D	I	A	W	I	N	F	B	N	Y	P	*
V	T	T	M	Z	F	P	K	W	G	D	K	Z	X	T	J	C	D	I	G	K	U	H	U	A	U	E	K	C	A	R	*

A simple division of 3 over 31 gives 0.097 with an inverse tan of 5.53 degrees. A scan of the internet doesn't return anything immediately obvious with regards this angle, but I note it here just in case it has some relevance.

Appendix 2. The Artists Signature

There may well be a masonic connection here as Masons refer to God as the Great Architect of the Universe, but observers outside of the craft also note that the G so often seen in their symbolism seems to be interchangeable in meaning between God and Geometry. Either way it's nice to have been introduced to the puzzle via Dan Brown's novel 'The Lost Symbol' and notice this multiline anagram in K1 stating 'G THE CANDLE OF IQ'. Whether or not this was intentional or not it's nice to think that Geometry gets a mention as being the light of IQ. Whether or not this has any relation to the clue in K3 about inserting the candle is conjecture, but it does occur right at the boundary between our coordinate and calendar blocks.

B	E	T	W	E	E	N	S	U	B	T	L	E	S	H	A	D	I	N	G	A	N	D	T	H	E	A	B	S	E	N	C	
E	O	F	L	I	G	H	T	L	I	E	S	T	H	E	N	U	A	N	C	E	O	F	I	Q	L	U	S	I	O	N		

Alternatively 'The Artists Signature' may well also be a direct reference to the symbols participating in Euler's Identity as outlined earlier.

Source Code

The source code provided is for a simple C# winforms project. No copyright of warranty is given, so feel free to customize as you wish. The project essentially began as a scratch pad to test these ideas, so please don't look for any elegance or consistency in the code as it is not written to any standard. There are still numerous bugs as well, but I'll endeavour to tidy things up and post corrected versions of the code base once I have the time. By toggling the clock button you can turn the binary clock off and on in order to speed the progression of the K3 decryption. Please note that the K3 decryption as it comes out letter by letter doesn't fall in the same grid positions as the conventional K3 decryption as yet, but this was purely down to time rather than design.

Play around with the various buttons to see what they do, but it's fairly self-explanatory apart from the 'Dials' button, which causes a graphics sub routine to render the relative position of the Moon with respect to the Earth at any given moment. If you create a new winforms project in something like C# express and copy this code over the top of the defaults then it should hopefully run off the bat. You'll have to add two new classes to the project called RowNavigation.cs and MatrixColourAssignments.cs respectively before copying the code in over the top.

Source Code for Form1.cs

```
using System;

using System.Collections.Generic;

using System.ComponentModel;

using System.Data;

using System.Drawing;

using System.Linq;

using System.Text;

using System.Threading.Tasks;

using System.Windows.Forms;

using System.Drawing.Drawing2D;

namespace CMechanics
{
    public partial class Form1 : Form
    {
        private double earthAngle = -1;

        private int earthAngleInt = 90;

        private double moonAngle = 90;

        private double moonAngleFloat = 90;

        // Random generator

        //private Random RandomGenerator = new Random();

        private int Radius = 50;

        private bool init = true;

        private bool isPositive = true;
```

```

private bool pass1 = true;

private bool bMoonOffset = false;

private int moonOffset = 99;

private int moonPhase = 0;

private Dictionary<int, TextBox> matrix = new Dictionary<int, TextBox>();

private Dictionary<int, rowPos> k3Instructions = new Dictionary<int, rowPos>();

private Dictionary<int, rowPos> rowInstructions = new Dictionary<int, rowPos>();

private Dictionary<int, rowPos> calInstructions = new Dictionary<int, rowPos>();

private Dictionary<int, int> phaseArray = new Dictionary<int, int>();

private int calCounter = 20;

private int matrixPos = 0;

private int earthCounter = 0;

private int Counter360 = 0;

private int moonCounter = 0;

private int squareCounterGlobal = 0;

private int k3Counter = 462;

private int lastK3Counter = 463;

private Color lastK3Colour = Color.LightBlue;

private Color lastColour = Color.CornflowerBlue;

private Color lastMoonColour = Color.CornflowerBlue;

private Color lastCalColour = Color.LightGreen;

private int lastMatrixValue = 0;

private int lastCalCounter = 20;

private bool is38 = true;

private int dayCounter = 1;

private int weekcounter = 0;

private int weekCounterAdj = 0;

private int monthCounter = 0;

private int sideralMonthsBinary = 0;

private int synodicMonthsBinary = 0;


private int hours = 0;

private int minutes = 0;

private int seconds = 0;

```

```

private Color lastDayColour;

public Form1()
{
    InitializeComponent();

    CreateHardware("CTR",40,40);

    CreateHardware("PTR", 845, 40);

    LoadCoordinateTraversalRowInstructions();

    LoadCalendarTraversalInstructions();

    LoadK3TraversalInstructions();

    LoadPhaseArray();

    //matrix[20].BackColor = Color.Yellow;
}

private bool bDisplayDials = false;
private System.Drawing.Bitmap bmp = null;
private bool bScreenCaptured = false;
private Bitmap b = null;
protected override void OnPaint(PaintEventArgs e)
{
    if (bDisplayDials)
    {
        int diameter = Radius * 2;

        e.Graphics.SmoothingMode = System.Drawing.Drawing2D.SmoothingMode.HighQuality;

        // Draw a filled rectangle to remove any traces of old dial lines
        e.Graphics.FillRectangle(new SolidBrush(BackColor), 0, 0, diameter, diameter);

        // Draw the circle
        e.Graphics.DrawEllipse(Pens.White, 720, 65, diameter, diameter);

        //
        e.Graphics.DrawEllipse(Pens.White, 720, 230, diameter, diameter);

        // Draw the randomly generated angle
        // If you look above, I defined the initial value to be -1
        if (earthAngle > -1)
        {

```

```

        int x_center = Radius;

        int y_center = Radius;

        int x_edge = (int) (Radius + Radius * Math.Cos(earthAngle));

        int y_edge = (int) (Radius - Radius * Math.Sin(earthAngle));

        int x_MoonEdge = (int) (Radius + Radius * Math.Cos(moonAngle));

        int y_MoonEdge = (int) (Radius - Radius * Math.Sin(moonAngle));

        e.Graphics.DrawLine(Pens.White, x_center + 720, y_center + 65, x_edge +
720, y_edge + 65);

        e.Graphics.DrawLine(Pens.White, x_center + 720, y_center + 230, x_MoonEdge
+ 720, y_MoonEdge + 230);

    }

    String drawString = "Earth Rotation";

    // Create font and brush.
    Font drawFont = new Font("Calibri Light", 16);
    SolidBrush drawBrush = new SolidBrush(Color.White);

    // Create point for upper-left corner of drawing.
    PointF drawPoint = new PointF((Radius + 655), (Radius - 16));

    // Draw string to screen.
    e.Graphics.DrawString(drawString, drawFont, drawBrush, drawPoint);

    String drawString2 = "Moon Position";

    // Create point for upper-left corner of drawing.
    PointF drawPoint2 = new PointF((Radius + 655), (Radius + 150));

    // Draw string to screen.
    e.Graphics.DrawString(drawString2, drawFont, drawBrush, drawPoint2);

```

```

        String drawString3 = "Degrees " + Counter360.ToString();

        // Create point for upper-left corner of drawing.
        PointF drawPoint3 = new PointF((Radius + 675), (Radius + 115));

        // Draw string to screen.
        e.Graphics.DrawString(drawString3, drawFont, drawBrush, drawPoint3);

        String drawString4 = "Moon Deg Rel"; // +moonAngleFloat.ToString();

        // Create point for upper-left corner of drawing.
        PointF drawPoint4 = new PointF((Radius + 655), (Radius + 280));

        // Draw string to screen.
        e.Graphics.DrawString(drawString4, drawFont, drawBrush, drawPoint4);

    }

    base.OnPaint(e);

    if (bScreenCaptured)
    {
        e.Graphics.DrawImage(b, new Point(150, 150));
    }
}

private void CreateHardware(string tag,int rightOffset, int topOffset)
{
    //Create the base hardware with no data
    TextBox[] tb = new TextBox[924];

    //Font font = new Font("Calibri Light", 12.0f);

    //Font font = new Font("Arial", 12.0f);

    Font font = new Font("Times", 12.0f);

```

```

int squareCounter = 0;

int rightCount = rightOffset;

SuspendLayout();

for (int i = 1; i < 29; i++)
{
    for (int j = 1; j < 34; j++)
    {
        tb[squareCounter] = new TextBox();

        tb[squareCounter].ForeColor = Color.White;

        tb[squareCounter].BackColor = Color.FromArgb(85, 101, 125);

        tb[squareCounter].ReadOnly = true;

        tb[squareCounter].Multiline = false;

        tb[squareCounter].TextAlign = HorizontalAlignment.Center;

        tb[squareCounter].Font = font;

        tb[squareCounter].Top = topOffset;

        tb[squareCounter].Tag = tag + "-" + i.ToString() + "-" + j.ToString();

        tb[squareCounter].Left = rightCount;

        tb[squareCounter].Width = 20;

        tb[squareCounter].Height = 10;

        tb[squareCounter].BorderStyle = BorderStyle.FixedSingle;

        this.Controls.Add(tb[squareCounter]);

        matrix.Add(squareCounterGlobal + 1, tb[squareCounter]);

        squareCounter++;

        squareCounterGlobal++;

        rightCount += 20;
    }

    if (i == 14)
    {
        topOffset += 50;
    }

    else
    {

```

```

        topOffset += 22;

    }

    rightCount = rightOffset;

}

ResumeLayout();

}

private void UpdateEarthAngle()
{
    earthAngleInt--;
    if (earthAngleInt == 0)
    {
        earthAngleInt = 360;
    }

    // Convert from degrees to radians:
    earthAngle = earthAngleInt * Math.PI / 180;

}

private void UpdateMoonAngle()
{
    moonAngleFloat = moonAngleFloat - 6.5;

    if(moonAngleFloat < 0)
    {
        moonAngleFloat = 360 - (moonAngleFloat - 90);
    }

    // Convert from degrees to radians:
    moonAngle = moonAngleFloat * Math.PI / 180;

}

```

```

private void LoadCoordinateTraversalRowInstructions()
{
    RowNavigation.LoadCoordinateTraversalRows(rowInstructions);

    return;
}

private void LoadK3TraversalInstructions()
{
    RowNavigation.LoadK3TraversalRows(k3Instructions);

    return;
}

private void LoadCalendarTraversalInstructions()
{
    RowNavigation.LoadCalendarTraversalRows(calInstructions);

    return;
}

private void LoadPhaseArray()
{
    //This loads the relative positions of the asterisks at the row ends
    //for the lunar month progression routine

    phaseArray.Add(1, 65);
    phaseArray.Add(2, 98);
    phaseArray.Add(3, 130);
    phaseArray.Add(4, 131);
    phaseArray.Add(5, 164);
    phaseArray.Add(6, 230);
    phaseArray.Add(7, 263);
    phaseArray.Add(8, 329);
    phaseArray.Add(9, 361);
}

```

```

        phaseArray.Add(10, 362);

        phaseArray.Add(11, 395);

        phaseArray.Add(12, 428);

        phaseArray.Add(13, 461);

        phaseArray.Add(14, 526);

        phaseArray.Add(15, 527);

        phaseArray.Add(16, 560);


        phaseArray.Add(17, 592);

        phaseArray.Add(18, 593);

        phaseArray.Add(19, 658);

        phaseArray.Add(20, 659);

        phaseArray.Add(21, 725);

        phaseArray.Add(22, 789);

        phaseArray.Add(23, 790);

        phaseArray.Add(24, 791);

        phaseArray.Add(25, 824);

        phaseArray.Add(26, 857);

        phaseArray.Add(27, 890);

        phaseArray.Add(28, 923);

    }

    private void OnTick()
    {
        //Runs everytime the timer fires

        k3Counter++;

        DoK3Update();


        earthCounter++;

        if (earthCounter == 181)
        {
            //Start Moon offset

```

```

        bMoonOffset = true;
    }

    if (bMoonOffset)
    {

        moonOffset++;

        if (moonOffset == 112)
        {
            bMoonOffset = false;

            //Counter360 = 360;

            if (pass1)
            {
                //Light up indicator

                matrix[106].BackColor = Color.White;

                //pass1 = false;

                moonOffset = 99;

                matrix[111].BackColor = lastMoonColour;

                decryptCounter++;

                matrix[1386 + decryptCounter].Text = matrix[k3Counter].Text;
            }
            else
            {
                int lastPhase;

                //dont highlight. Fill in moon phase

                matrix[106].BackColor = System.Drawing.SystemColors.ActiveCaption; ;

                moonPhase++;

                if (moonPhase == 29 || moonPhase == 1)
                {
                    //Update the sidereal moon month

                    siderealMonthsBinary++;

                    moonPhase = 1;

                    //matrix[phaseArray[28]].BackColor = Color.LightGreen;

```

```

        lastPhase = 28;
    }
    else
    {
        lastPhase = moonPhase - 1;
    }

    matrix[phaseArray[lastPhase]].BackColor = Color.LightGreen;
    matrix[phaseArray[moonPhase]].BackColor = Color.Yellow;

    //Tidy up final
    matrix[111].BackColor = lastMoonColour;

    moonOffset = 99;

    //pass1 = true;

    decryptCounter++;

    matrix[1386 + decryptCounter].Text = matrix[k3Counter].Text;
}
}
else
{
    if (moonOffset != 100)
    {
        matrix[moonOffset - 1].BackColor = lastMoonColour;
    }

    if(moonOffset == 106)
    {
        moonOffset = 107; //Skip the decimal point block
    }

    if(moonOffset == 107)
    {
        matrix[105].BackColor = lastMoonColour;
    }

    lastMoonColour = matrix[moonOffset].BackColor;

    matrix[moonOffset].BackColor = Color.Yellow;

```

```

        return;
    }

}

else
{
    Counter360++;
    if (Counter360 == 361)
    {
        //Update the day counter
        int lastDayCounter = dayCounter;
        dayCounter++;
        if(dayCounter ==8)
        {
            DoWeeks();

            dayCounter = 1;
        }

        matrix[297 + lastDayCounter].BackColor = lastDayColour;
        lastDayColour = matrix[297 + dayCounter].BackColor;
        matrix[297 + dayCounter].BackColor = Color.Yellow;

        Counter360 = 1;
    }
}

if (earthCounter == 192)
{
    DoRotation();
    earthCounter = 0;
    UpdateMoonAngle();
    if (pass1)
    {

```

```

        //Light up the ? indicator

        pass1 = false;

    }

    else

    {

        calCounter++;

        DoCalendar();

        pass1 = true;

    }

    Invalidate(); //Forces form to redraw

    return;

    //Update the day
}

UpdateEarthAngle();

// Update the dial line by calling the paint event, otherwise known as calling the
Invalidate() method

Invalidate();

DoRotation();

}

private int weekBase = 442; //Position in matrix for the start of the week count
private void DoWeeks()
{
    //Increment the week counter

    weekcounter++;

    weekCounterAdj++;

    if (weekcounter == 53)
    {
        weekcounter = 1;
    }
}

```

```

    }

    if (weekcounter == 13)
    {
        weekCounterAdj = 1;

        weekBase = 409;
    }

    if (weekcounter == 26)
    {
        weekCounterAdj = 1;

        weekBase = 376;
    }

    if (weekcounter == 39)
    {
        weekCounterAdj = 1;

        weekBase = 343;
    }

    if (weekcounter == 52)
    {
        weekCounterAdj = 1;

        weekBase = 310;
    }

    matrix[weekBase - weekCounterAdj].BackColor = Color.YellowGreen;
}

private int decryptCounter = 0;

private int last4MinuteCount = 0;

private void timer1_Tick(object sender, EventArgs e)
{
    //If the binary clock is enabled then update the coordinate traversal
    //every 4 minutes

    if (bBinaryClockIsOn)
    {
        DoBinaryClock();
    }
}

```

```

        if ((binMins % 4) == 0 && binMins != 0)
        {
            if (last4MinuteCount != binMins)
            {
                last4MinuteCount = binMins;

                OnTick();
            }
        }
    }
else
{
    OnTick();
}
}

private int binHours = 0;
private int binMins = 0;
private int binSecs = 0;
//5 Bit Cal
private int binDayOfWeek = 1;
private int binMonth = 1;
private int binDayOfMonth = 1;

private const int bit1 = 0;
private const int bit2 = 1;
private const int bit3 = 2;
private const int bit4 = 3;
private const int bit5 = 4;
private const int bit6 = 5;
private bool bInLeapYear = false;
private int binYear = 2017;
private void DoDayOfMonth(int binDayOfMonth, int binMonth)
{

```

```

switch (binMonth)
{
    case 1:
    case 3:
    case 5:
    case 7:
    case 8:
    case 10:
    case 12:
        {
            if (binDayOfMonth > 31)
            {
                binDayOfMonth = 1;
                binMonth++;
                if (binMonth > 12)
                {
                    binYear++;
                }
            }
            break;
        }
    case 4:
    case 6:
    case 9:
    case 11:
        {
            if (binDayOfMonth > 30)
            {
                binDayOfMonth = 1;
                binMonth++;
            }
            break;
        }
}

```

```

        case 2:
        {
            //Case for feb
            if (binDayOfMonth > 28)
            {
                if (binLeapYear == false)
                {
                    binDayOfMonth = 1;
                    binMonth++;
                }
                else
                {
                    if (binDayOfMonth > 29)
                    {
                        binDayOfMonth = 1;
                        binMonth++;
                    }
                }
            }
            break;
        }
        default:
        {
            //Should never be here
            break;
        }
    }
}

private void DoBinaryClock()
{
    //update current time
    binSecs += 10;

```

```

if (binSecs > 60) //Using greater than allows us to speed up via += > 1
{
    binSecs = 1;
    binMins++;
    if (binMins == 61)
    {
        binMins = 1;
        binHours++;
        if (binHours == 24)
        {
            binHours = 0;
            binDayOfWeek++;
            binDayOfMonth++;
            if (binDayOfWeek > 7)
            {
                binDayOfWeek = 1;
            }
            DoDayOfMonth(binDayOfMonth, binMonth);
        }
    }
}

string binarySeconds = Convert.ToString(binSecs, 2).PadLeft(6, '0');
DoBinaryTime(binarySeconds, 199,6);

string binaryMinutes = Convert.ToString(binMins, 2).PadLeft(6, '0');
DoBinaryTime(binaryMinutes, 166,6);

string binaryHours = Convert.ToString(binHours, 2).PadLeft(6, '0');
DoBinaryTime(binaryHours, 133,6);

string binaryDayOfWeek = Convert.ToString(binDayOfWeek, 2).PadLeft(6, '0');
DoBinaryTime(binaryDayOfWeek, 206,5);

```

```

        string binaryDayOfMonth = Convert.ToString(binDayOfMonth, 2).PadLeft(6, '0');
        DoBinaryTime(binaryDayOfMonth, 173, 5);

        string binaryMonth = Convert.ToString(binMonth, 2).PadLeft(6, '0');
        DoBinaryTime(binaryMonth, 140, 5);

    }

    private void DoBinaryTime(string binary, int startBlock, int bitPrecsion)
    {

        char[] lights = binary.ToArray<char>();

        if (lights[bit6] == '1')
        {
            matrix[startBlock + 0].BackColor = Color.Yellow;
        }
        else
        {
            matrix[startBlock + 0].BackColor = Color.FromArgb(85, 101, 125);
        }
        if (lights[bit5] == '1')
        {
            matrix[startBlock + 1].BackColor = Color.Yellow;
        }
        else
        {
            matrix[startBlock + 1].BackColor = Color.FromArgb(85, 101, 125);
        }
        if (lights[bit4] == '1')
        {
            matrix[startBlock + 2].BackColor = Color.Yellow;
        }
        else
    }

```

```

        {
            matrix[startBlock + 2].BackColor = Color.FromArgb(85, 101, 125);
        }
        if (lights[bit3] == '1')
        {
            matrix[startBlock + 3].BackColor = Color.Yellow;
        }
        else
        {
            matrix[startBlock + 3].BackColor = Color.FromArgb(85, 101, 125);
        }
        if (lights[bit2] == '1')
        {
            matrix[startBlock + 4].BackColor = Color.Yellow;
        }
        else
        {
            matrix[startBlock + 4].BackColor = Color.FromArgb(85, 101, 125);
        }
        if (bitPrecsion == 6)
        {
            if (lights[bit1] == '1')
            {
                matrix[startBlock + 5].BackColor = Color.Yellow;
            }
            else
            {
                matrix[startBlock + 5].BackColor = Color.FromArgb(85, 101, 125);
            }
        }
    }

    private void DoK3Update()
    {

```

```

        if (k3Instructions.ContainsKey(k3Counter))
        {
            k3Counter = k3Instructions[k3Counter].nextRow;

            //isPositive = rowInstructions[earthCounter].positive;
        }

        if (k3Counter != 1)
        {
            matrix[lastK3Counter].BackColor = lastK3Colour;
        }
        else
        {
            matrix[lastK3Counter].BackColor = Color.LightGreen;
        }

        lastK3Colour = matrix[k3Counter].BackColor;
        matrix[k3Counter].BackColor = Color.Yellow;
        lastK3Counter = k3Counter;
    }

    private void DoCalendar()
    {
        if (calInstructions.ContainsKey(calCounter))
        {
            calCounter = calInstructions[calCounter].nextRow;

            //isPositive = rowInstructions[earthCounter].positive;
        }

        if (calCounter != 20)
        {
            matrix[lastCalCounter].BackColor = lastCalColour;
        }
        else
        {
            matrix[lastCalCounter].BackColor = Color.LightGreen;
        }

        lastCalColour = matrix[calCounter].BackColor;
        matrix[calCounter].BackColor = Color.Yellow;
    }

```

```

        lastCalCounter = calCounter;

    }

    private void DoRotation()
    {
        if (rowInstructions.ContainsKey(earthCounter))
        {
            matrixPos = rowInstructions[earthCounter].nextRow;

            isPositive = rowInstructions[earthCounter].positive;
        }

        if (isPositive)
        {
            matrixPos++;
        }
        else
        {
            matrixPos--;
        }

        if (init)
        {
            // matrix[lastMatrixValue].BackColor = lastColour;

            matrix[matrixPos].BackColor = MatrixColourAssignments.coordinateTraversal;

            lastMatrixValue = matrixPos;

            init = false;
        }
        else
        {
            matrix[lastMatrixValue].BackColor = lastColour;

            lastColour = matrix[matrixPos].BackColor;

            matrix[matrixPos].BackColor = MatrixColourAssignments.coordinateTraversal;

            lastMatrixValue = matrixPos;

            //lastMatrixValue++;
        }
    }
}

```

```

    }

}

private bool bShowLetters = false;

private void button1_Click(object sender, EventArgs e)
{
    //Fill the ct with letters

    if (bShowLetters == false)
    {
        string ct = "";

        ct = ct + "EMUFPHZLRFAXYUSDJKZLDKRNSHGNFIVJ ";
        ct = ct + "YQTQUXQBQVYUUVLLTREVJYQTMKYRDMFD* ";
        ct = ct + "VFPJUDEEHZWETZYVGWHKKQETGFQJNCE* ";
        ct = ct + "GGWHKK?DQMC PFQZDQMMIAGPFHXHQLG** ";
        ct = ct + "TIMVMZJANQLVKQEDAGDVFRPJUNGEUNA* ";
        ct = ct + "QZGZLECGYUXUEENJTBBLBQCRBTBJDFHRR ";
        ct = ct + "YIZETKZEMVDUFKSJHKFWHKUWQLSZFTI* ";
        ct = ct + "HHDDDUVH?DWKBFUFPPWNTDFIYCUQZERE* ";
        ct = ct + "EVLDFEZZMOQQJLTTUGSYQPFEUNLAVIDX ";
        ct = ct + "FLGGTEZ?FKZBSFDQVGOGIPUFXXHHRKF* ";
        ct = ct + "FHQNTGPPUAECNUVPDJMQCLQUMUNEDFQ** ";
        ct = ct + "ELZZVRRGKFFVOEEXBDMVPNFQXEZLGRE* ";
        ct = ct + "DNQFMPNZGLFLPMRJQYALMGNUVPDXVKP* ";
        ct = ct + "DQUMEBEDMHDAFMJGZNUPLGEWJLLAETG* ";
        ct = ct + "ENDYAHROHNLSRHEOCPTIOIBIDYSHNAIA ";
        ct = ct + "CHTNREYULDSLSSLNOHSNOSMRWXMNE** ";
        ct = ct + "TPRNGATIHNRARPESLNNELEBLPIIACAE* ";
        ct = ct + "WMTWNDITEENRAHCTENEUDRETNHAE** ";
        ct = ct + "TFOLSEDTIWENHAEIOYTEYQHEENCTAYCR ";
        ct = ct + "EIFTBRSPAMHHEWENATAMATEGYEERLB** ";
        ct = ct + "TEEFOASFIOTUETUAEOTOARMAEERTNRTI ";
        ct = ct + "BSEDDNIAAHTTMSTEWPIEROAGRIEFEB* ";
        ct = ct + "AECTDDHILCEIHSITEGOEAOSSDDRYDLORIT";
    }
}

```

```

        ct = ct + "RKLMLEHAGTDHARDPNEOHMGFMFEUHE*** ";
        ct = ct + "ECDMRIPFEIMEHNLSSTTRTVDOHW?OBKR* ";
        ct = ct + "UOXOGHULBSOLIFBBWFLRVQQPRNGKSSO* ";
        ct = ct + "TWTQJSJSSEKZZWATJKLUDIAWINFBNYP* ";
        ct = ct + "VTMZFPKWGDKZXTJCDIGKUHUAUEKCAR* ";

        char[] source = ct.ToArray<char>();

        for (int i = 0; i < 924; i++)
        {
            matrix[i + 1].Text = source[i].ToString();
        }

        bShowLetters = true;
    }
    else
    {
        for (int i = 0; i < 924; i++)
        {
            matrix[i + 1].Text = "";
        }

        bShowLetters = false;
    }
}

private void button2_Click(object sender, EventArgs e)
{
    MatrixColourAssignments.ColourCoordinateBlock38(matrix);
}

private void button3_Click(object sender, EventArgs e)
{
    MatrixColourAssignments.ColourCoordinateBlock57(matrix);
}

private void button4_Click(object sender, EventArgs e)

```

```

{
    MatrixColourAssignments.ColourCoordinateBlock6P5(matrix);
}

private void button5_Click(object sender, EventArgs e)
{
    MatrixColourAssignments.ColourCoordinateBlock77(matrix);
}

private void button7_Click(object sender, EventArgs e)
{
    MatrixColourAssignments.ColourCoordinateBlock36(matrix);
}

private void button6_Click(object sender, EventArgs e)
{
    MatrixColourAssignments.ColourCoordinateBlock8(matrix);
}

private void button11_Click(object sender, EventArgs e)
{
    MatrixColourAssignments.ColourCoordinateBlock3(matrix);

    /*
    for (int i = 241; i < 244; i++)
    {
        matrix[i].BackColor = Color.FromArgb(181, 93, 226);
    }*/
}

private void button10_Click(object sender, EventArgs e)
{
    MatrixColourAssignments.ColourCoordinateBlock12(matrix);
}

```

```

private void button9_Click(object sender, EventArgs e)
{
    MatrixColourAssignments.ColourCoordinateBlock7(matrix);
}

private void button12_Click(object sender, EventArgs e)
{
    MatrixColourAssignments.ColourCoordinateBlock4(matrix);
}

private void button13_Click(object sender, EventArgs e)
{
    MatrixColourAssignments.ColourCoordinateBlock48(matrix);
}

private void button8_Click(object sender, EventArgs e)
{
    MatrixColourAssignments.ColourCoordinateBlock365(matrix);
}

private bool timerEnabled = false;
private void button14_Click(object sender, EventArgs e)
{
    if (timerEnabled == false)
    {
        timer1.Enabled = true;
        timerEnabled = true;
    }
    else
    {
        timer1.Enabled = false;
        timerEnabled = false;
    }
}

```

```

private void button15_Click(object sender, EventArgs e)
{
    OnTick();

    Invalidate();
}

private void button16_Click(object sender, EventArgs e)
{
    for (int i = 0; i < 924; i++)
    {
        matrix[i + 1].BorderStyle = BorderStyle.None;
    }
}

private void button17_Click(object sender, EventArgs e)
{
    string pt = "BETWEENSUBTLESHADINGANDTHEABSENC EOFLIGHTLIESTHENUANCEOFIQLUSION* ";

    char[] source = pt.ToArray<char>();

    for (int i = 924; i < pt.Length + 924; i++)
    {
        matrix[i + 1].Text = source[i-924].ToString();
    }
}

private void button18_Click(object sender, EventArgs e)
{
    string tt = " ABCDEFGHIJKLMNOPQRSTUVWXYZABCD ";

    tt = tt + "AKRYPTOSABCDEFGHIJLMNQUVWXZKRYPT ";
    tt = tt + "BRYPTOSABCDEFGHIJLMNQUVWXZKRYPT ";
    tt = tt + "CYPTOSABCDEFGHIJLMNQUVWXZKRYPTO ";
    tt = tt + "DPTOSABCDEFGHIJLMNQUVWXZKRYPTOS ";
    tt = tt + "ETOSABCDEFGHIJLMNQUVWXZKRYPTOSA ";
    tt = tt + "FOSABCDEFGHIJLMNQUVWXZKRYPTOSAB ";
    tt = tt + "GSABCDEFGHIJLMNQUVWXZKRYPTOSABC ";
}

```

```

        tt = tt + "HABCDEFGH IJLMNQUVWXZKRYPTOSABCD ";
        tt = tt + "I BCDEFGH IJLMNQUVWXZKRYPTOSABCDE ";
        tt = tt + "J CDEFGH IJLMNQUVWXZKRYPTOSABCDEF ";
        tt = tt + "K DEFGH IJLMNQUVWXZKRYPTOSABCDEFG ";
        tt = tt + "L EFGH IJLMNQUVWXZKRYPTOSABCDEFGH ";
        tt = tt + "M FGH IJLMNQUVWXZKRYPTOSABCDEFGHI ";

        tt = tt + "N GH IJLMNQUVWXZKRYPTOSABCDEFGHIJL ";
        tt = tt + "O HIJLMNQUVWXZKRYPTOSABCDEFGHIJL ";
        tt = tt + "P IJLMNQUVWXZKRYPTOSABCDEFGHIJLM ";
        tt = tt + "Q JLMNQUVWXZKRYPTOSABCDEFGHIJLMN ";
        tt = tt + "R LMNQUVWXZKRYPTOSABCDEFGHIJLMNQ ";
        tt = tt + "S MNQUVWXZKRYPTOSABCDEFGHIJLMNQU ";
        tt = tt + "T NQUVWXZKRYPTOSABCDEFGHIJLMNQUV ";
        tt = tt + "U QUVWXZKRYPTOSABCDEFGHIJLMNQUVW ";
        tt = tt + "V UVWXZKRYPTOSABCDEFGHIJLMNQUVWX ";
        tt = tt + "W VWXZKRYPTOSABCDEFGHIJLMNQUVWXZ ";
        tt = tt + "X WXZKRYPTOSABCDEFGHIJLMNQUVWXZK ";
        tt = tt + "Y XZKRYPTOSABCDEFGHIJLMNQUVWXZKR ";
        tt = tt + "Z ZKRYPTOSABCDEFGHIJLMNQUVWXZKRY ";
        tt = tt + " ABCDEFGHIJKLMNOPQRSTUVWXYZABCD ";

        char[] source = tt.ToArray<char>();

        for (int i = 924; i < tt.Length + 924; i++)
        {
            matrix[i + 1].Text = source[i - 924].ToString();
        }
    }

    private void button19_Click(object sender, EventArgs e)
    {
        string k2 = "ITWASTOTALLYINVISIBLEHOWSTHATPO* ";
        k2 = k2 + "SSIBLE?THEYUSEDTHEEARTHSMAGNET** ";
        k2 = k2 + "ICFIELDXTHEINFORMATIONWASGATHER* ";
    }

```

```

k2 = k2 + "EDANDTRANSMITTEDUNDERGRUUNDTOANU ";
k2 = k2 + "NKNOWNLOCATIONXDOESLANGLEYKNOWA* ";
k2 = k2 + "BOUTHIS?THEYSHOULDITSBURIEDOUT* ";
k2 = k2 + "THERESOMEWHEREXWHOKNOWSTHEEXACTL ";
k2 = k2 + "OCATION?ONLYWWTHISWASHISLASTMES* ";
k2 = k2 + "SAGEXTHIRTYEIGHTDEGREESFIFTYSE** ";
k2 = k2 + "VENMINUTESEXPOINTFIVESECONDSNO* ";
k2 = k2 + "RTHSEVENTYSEVENDEGREESEIGHTMINU* ";
k2 = k2 + "TESFORTYFOURSECONDSWESTIDBYRO";
k2 = k2 + "WS*";

char[] source = k2.ToArray<char>();
for (int i = 990; i < k2.Length + 990; i++)
{
    matrix[i + 1].Text = source[i - 990].ToString();
}

private void button20_Click(object sender, EventArgs e)
{
    if (bDisplayDials == false)
    {
        bDisplayDials = true;
    }
    else
    {
        bDisplayDials = false;
    }
    Invalidate();
}

private void button21_Click(object sender, EventArgs e)
{
    for (int i = 0; i < 32; i++)

```

```

        {
            matrix[i + 1].BackColor = Color.FromArgb(54, 83, 130);
        }
        for (int i = 33; i < 65; i++)
        {
            matrix[i + 1].BackColor = Color.FromArgb(54, 83, 130);
        }
    }

private void button22_Click(object sender, EventArgs e)
{
    int rowCount = 0;
    for (int i = 66; i < 461; i++)
    {
        rowCount++;
        if (rowCount == 33)
        {
            rowCount = 0;
        }
        else
        {
            matrix[i + 1].BackColor = Color.FromArgb(59, 100, 165);
        }
    }
}

private void button23_Click(object sender, EventArgs e)
{
    int rowCount = 0;
    for (int i = 462; i < 819; i++)
    {
        rowCount++;
        if (rowCount == 33)
        {

```

```

        rowCount = 0;

    }

    else

    {

        matrix[i + 1].BackColor = Color.FromArgb(79, 123, 193);

    }

}

matrix[759].BackColor = Color.FromArgb(79, 123, 193);
}

```

```

private void button24_Click(object sender, EventArgs e)
{
    string k3 = "SLOWLYDESPARATLYSLOWLYTHEREMAINS ";
    k3 = k3 + "OFPASSAGEDEBRISTHATENCUMBEREDT** ";
    k3 = k3 + "HELOWERPARTOFTHEDOORWAYWASREMOV* ";
    k3 = k3 + "EDWITHTREMBLINGHANDSIMADEATINY** ";
    k3 = k3 + "BREACHINTHEUPPERLEFTHANDCORNERAN ";
    k3 = k3 + "DTHENWIDENINGTHEHOLEALITTLEIIN** ";
    k3 = k3 + "SERTEDTHECANDLEANDPEEREDINTHEHOT ";
    k3 = k3 + "AIRESCAPINGFROMTHECHAMBERCAUSED* ";
    k3 = k3 + "THEFLAMETOFLICKERBUTPRESENTLYDETA";
    k3 = k3 + "ILSOFThEROOMWITHINEMERGEDFROM*** ";
    k3 = k3 + "THEMISTXCANYOUSEEANYTHINGQ?";
    char[] source = k3.ToArray<char>();
    for (int i = 1386; i < k3.Length + 1386; i++)
    {
        matrix[i + 1].Text = source[i - 1386].ToString();
    }
}

```

```

private void button25_Click(object sender, EventArgs e)
{
    int rowCount = 0;

```

```

        for (int i = 819; i < 824; i++)
        {
            matrix[i + 1].BackColor = Color.FromArgb(107, 71, 119);
        }

        for (int i = 825; i < 923; i++)
        {
            rowCount++;

            if (rowCount == 33)
            {
                rowCount = 0;
            }

            else
            {
                matrix[i + 1].BackColor = Color.FromArgb(107, 71, 119);
            }
        }
    }

    private void Form1_Load(object sender, EventArgs e)
    {
        return;
    }

    private bool bBinaryClockIsOn = false;

    private void btnClock_Click(object sender, EventArgs e)
    {
        if (bBinaryClockIsOn)
        {
            bBinaryClockIsOn = false;
        }

        else
        {
            bBinaryClockIsOn = true;
        }
    }
}

```

```
    }  
}
```

Source Code for Form1.Designer.cs

```
namespace CMechanics  
{  
    partial class Form1  
    {  
        /// <summary>  
        /// Required designer variable.  
        /// </summary>  
        private System.ComponentModel.IContainer components = null;  
  
        /// <summary>  
        /// Clean up any resources being used.  
        /// </summary>  
        /// <param name="disposing">true if managed resources should be disposed; otherwise,  
false.</param>  
        protected override void Dispose(bool disposing)  
        {  
            if (disposing && (components != null))  
            {  
                components.Dispose();  
            }  
            base.Dispose(disposing);  
        }  
  
        #region Windows Form Designer generated code  
  
        /// <summary>  
        /// Required method for Designer support - do not modify  
        /// the contents of this method with the code editor.  
    }  
}
```

```

/// </summary>

private void InitializeComponent()
{
    this.components = new System.ComponentModel.Container();

    this.timer1 = new System.Windows.Forms.Timer(this.components);

    this.btnPopulateCT = new System.Windows.Forms.Button();

    this.btn38 = new System.Windows.Forms.Button();

    this.btn57 = new System.Windows.Forms.Button();

    this.btn6p5 = new System.Windows.Forms.Button();

    this.btn77 = new System.Windows.Forms.Button();

    this.btn8 = new System.Windows.Forms.Button();

    this.btn44 = new System.Windows.Forms.Button();

    this.btnCalendar = new System.Windows.Forms.Button();

    this.btn7 = new System.Windows.Forms.Button();

    this.btn12 = new System.Windows.Forms.Button();

    this.btn3 = new System.Windows.Forms.Button();

    this.btn4 = new System.Windows.Forms.Button();

    this.btn48 = new System.Windows.Forms.Button();

    this.btnStart = new System.Windows.Forms.Button();

    this.btnStep = new System.Windows.Forms.Button();

    this.btnBorder = new System.Windows.Forms.Button();

    this.btnK1 = new System.Windows.Forms.Button();

    this.btnTableau = new System.Windows.Forms.Button();

    this.btnK2 = new System.Windows.Forms.Button();

    this.btnShowDials = new System.Windows.Forms.Button();

    this.button21 = new System.Windows.Forms.Button();

    this.button22 = new System.Windows.Forms.Button();

    this.button23 = new System.Windows.Forms.Button();

    this.btnK3 = new System.Windows.Forms.Button();

    this.button25 = new System.Windows.Forms.Button();

    this.btnClock = new System.Windows.Forms.Button();

    this.SuspendLayout();

    //

    // timer1

```

```

//
this.timer1.Interval = 1;

this.timer1.Tick += new System.EventHandler(this.timer1_Tick);
//

// btnPopulateCT
//

this.btnPopulateCT.Location = new System.Drawing.Point(12, 15);

this.btnPopulateCT.Name = "btnPopulateCT";

this.btnPopulateCT.Size = new System.Drawing.Size(91, 51);

this.btnPopulateCT.TabIndex = 0;

this.btnPopulateCT.Text = "Fill";

this.btnPopulateCT.UseVisualStyleBackColor = true;

this.btnPopulateCT.Click += new System.EventHandler(this.button1_Click);
//

// btn38
//

this.btn38.Location = new System.Drawing.Point(161, 15);

this.btn38.Name = "btn38";

this.btn38.Size = new System.Drawing.Size(84, 51);

this.btn38.TabIndex = 1;

this.btn38.Text = "38";

this.btn38.UseVisualStyleBackColor = true;

this.btn38.Click += new System.EventHandler(this.button2_Click);
//

// btn57
//

this.btn57.Location = new System.Drawing.Point(251, 15);

this.btn57.Name = "btn57";

this.btn57.Size = new System.Drawing.Size(77, 51);

this.btn57.TabIndex = 2;

this.btn57.Text = "57";

this.btn57.UseVisualStyleBackColor = true;

this.btn57.Click += new System.EventHandler(this.button3_Click);
//

```

```

// btn6p5
//
this.btn6p5.Location = new System.Drawing.Point(334, 15);
this.btn6p5.Name = "btn6p5";
this.btn6p5.Size = new System.Drawing.Size(76, 51);
this.btn6p5.TabIndex = 3;
this.btn6p5.Text = "6.5";
this.btn6p5.UseVisualStyleBackColor = true;
this.btn6p5.Click += new System.EventHandler(this.button4_Click);
//
// btn77
//
this.btn77.Location = new System.Drawing.Point(416, 15);
this.btn77.Name = "btn77";
this.btn77.Size = new System.Drawing.Size(78, 51);
this.btn77.TabIndex = 4;
this.btn77.Text = "77";
this.btn77.UseVisualStyleBackColor = true;
this.btn77.Click += new System.EventHandler(this.button5_Click);
//
// btn8
//
this.btn8.Location = new System.Drawing.Point(500, 15);
this.btn8.Name = "btn8";
this.btn8.Size = new System.Drawing.Size(85, 51);
this.btn8.TabIndex = 5;
this.btn8.Text = "8";
this.btn8.UseVisualStyleBackColor = true;
this.btn8.Click += new System.EventHandler(this.button6_Click);
//
// btn44
//
this.btn44.Location = new System.Drawing.Point(591, 15);
this.btn44.Name = "btn44";

```

```

this.btn44.Size = new System.Drawing.Size(85, 51);

this.btn44.TabIndex = 6;

this.btn44.Text = "44";

this.btn44.UseVisualStyleBackColor = true;

this.btn44.Click += new System.EventHandler(this.button7_Click);

//

// btnCalendar

//

this.btnCalendar.Location = new System.Drawing.Point(1146, 15);

this.btnCalendar.Name = "btnCalendar";

this.btnCalendar.Size = new System.Drawing.Size(91, 51);

this.btnCalendar.TabIndex = 7;

this.btnCalendar.Text = "365";

this.btnCalendar.UseVisualStyleBackColor = true;

this.btnCalendar.Click += new System.EventHandler(this.button8_Click);

//

// btn7

//

this.btn7.Location = new System.Drawing.Point(682, 15);

this.btn7.Name = "btn7";

this.btn7.Size = new System.Drawing.Size(84, 51);

this.btn7.TabIndex = 8;

this.btn7.Text = "7";

this.btn7.UseVisualStyleBackColor = true;

this.btn7.Click += new System.EventHandler(this.button9_Click);

//

// btn12

//

this.btn12.Location = new System.Drawing.Point(772, 15);

this.btn12.Name = "btn12";

this.btn12.Size = new System.Drawing.Size(83, 51);

this.btn12.TabIndex = 9;

this.btn12.Text = "12";

this.btn12.UseVisualStyleBackColor = true;

```

```

this.btn12.Click += new System.EventHandler(this.button10_Click);

//

// btn3

//

this.btn3.Location = new System.Drawing.Point(861, 15);

this.btn3.Name = "btn3";

this.btn3.Size = new System.Drawing.Size(87, 51);

this.btn3.TabIndex = 10;

this.btn3.Text = "3";

this.btn3.UseVisualStyleBackColor = true;

this.btn3.Click += new System.EventHandler(this.button11_Click);

//

// btn4

//

this.btn4.Location = new System.Drawing.Point(954, 15);

this.btn4.Name = "btn4";

this.btn4.Size = new System.Drawing.Size(82, 51);

this.btn4.TabIndex = 11;

this.btn4.Text = "4";

this.btn4.UseVisualStyleBackColor = true;

this.btn4.Click += new System.EventHandler(this.button12_Click);

//

// btn48

//

this.btn48.Location = new System.Drawing.Point(1042, 15);

this.btn48.Name = "btn48";

this.btn48.Size = new System.Drawing.Size(86, 51);

this.btn48.TabIndex = 12;

this.btn48.Text = "48";

this.btn48.UseVisualStyleBackColor = true;

this.btn48.Click += new System.EventHandler(this.button13_Click);

//

// btnStart

//

```

```

this.btnStart.Location = new System.Drawing.Point(12, 85);

this.btnStart.Name = "btnStart";

this.btnStart.Size = new System.Drawing.Size(91, 51);

this.btnStart.TabIndex = 15;

this.btnStart.Text = "Go";

this.btnStart.UseVisualStyleBackColor = true;

this.btnStart.Click += new System.EventHandler(this.button14_Click);

//

// btnStep

//

this.btnStep.Location = new System.Drawing.Point(12, 176);

this.btnStep.Name = "btnStep";

this.btnStep.Size = new System.Drawing.Size(91, 51);

this.btnStep.TabIndex = 18;

this.btnStep.Text = "Step";

this.btnStep.UseVisualStyleBackColor = true;

this.btnStep.Click += new System.EventHandler(this.button15_Click);

//

// btnBorder

//

this.btnBorder.Location = new System.Drawing.Point(12, 272);

this.btnBorder.Name = "btnBorder";

this.btnBorder.Size = new System.Drawing.Size(91, 51);

this.btnBorder.TabIndex = 19;

this.btnBorder.Text = "Bor";

this.btnBorder.UseVisualStyleBackColor = true;

this.btnBorder.Click += new System.EventHandler(this.button16_Click);

//

// btnK1

//

this.btnK1.Location = new System.Drawing.Point(12, 469);

this.btnK1.Name = "btnK1";

this.btnK1.Size = new System.Drawing.Size(91, 51);

this.btnK1.TabIndex = 20;

```

```

this.btnK1.Text = "K1";

this.btnK1.UseVisualStyleBackColor = true;

this.btnK1.Click += new System.EventHandler(this.button17_Click);

//

// btnTableau

//

this.btnTableau.Location = new System.Drawing.Point(12, 367);

this.btnTableau.Name = "btnTableau";

this.btnTableau.Size = new System.Drawing.Size(91, 51);

this.btnTableau.TabIndex = 21;

this.btnTableau.Text = "Tab";

this.btnTableau.UseVisualStyleBackColor = true;

this.btnTableau.Click += new System.EventHandler(this.button18_Click);

//

// btnK2

//

this.btnK2.Location = new System.Drawing.Point(12, 544);

this.btnK2.Name = "btnK2";

this.btnK2.Size = new System.Drawing.Size(91, 51);

this.btnK2.TabIndex = 22;

this.btnK2.Text = "K2";

this.btnK2.UseVisualStyleBackColor = true;

this.btnK2.Click += new System.EventHandler(this.button19_Click);

//

// btnShowDials

//

this.btnShowDials.Location = new System.Drawing.Point(12, 689);

this.btnShowDials.Name = "btnShowDials";

this.btnShowDials.Size = new System.Drawing.Size(91, 51);

this.btnShowDials.TabIndex = 23;

this.btnShowDials.Text = "Dials";

this.btnShowDials.UseVisualStyleBackColor = true;

this.btnShowDials.Click += new System.EventHandler(this.button20_Click);

//

```

```

// button21
//
this.button21.Location = new System.Drawing.Point(12, 770);
this.button21.Name = "button21";
this.button21.Size = new System.Drawing.Size(91, 51);
this.button21.TabIndex = 24;
this.button21.Text = "k1c";
this.button21.UseVisualStyleBackColor = true;
this.button21.Click += new System.EventHandler(this.button21_Click);
//
// button22
//
this.button22.Location = new System.Drawing.Point(12, 827);
this.button22.Name = "button22";
this.button22.Size = new System.Drawing.Size(91, 51);
this.button22.TabIndex = 25;
this.button22.Text = "k2c";
this.button22.UseVisualStyleBackColor = true;
this.button22.Click += new System.EventHandler(this.button22_Click);
//
// button23
//
this.button23.Location = new System.Drawing.Point(12, 907);
this.button23.Name = "button23";
this.button23.Size = new System.Drawing.Size(91, 51);
this.button23.TabIndex = 26;
this.button23.Text = "k3c";
this.button23.UseVisualStyleBackColor = true;
this.button23.Click += new System.EventHandler(this.button23_Click);
//
// btnK3
//
this.btnK3.Location = new System.Drawing.Point(12, 614);
this.btnK3.Name = "btnK3";

```

```

this.btnK3.Size = new System.Drawing.Size(91, 51);

this.btnK3.TabIndex = 27;

this.btnK3.Text = "K3";

this.btnK3.UseVisualStyleBackColor = true;

this.btnK3.Click += new System.EventHandler(this.button24_Click);

//

// button25

//

this.button25.Location = new System.Drawing.Point(12, 972);

this.button25.Name = "button25";

this.button25.Size = new System.Drawing.Size(91, 51);

this.button25.TabIndex = 28;

this.button25.Text = "k4c";

this.button25.UseVisualStyleBackColor = true;

this.button25.Click += new System.EventHandler(this.button25_Click);

//

// btnClock

//

this.btnClock.Location = new System.Drawing.Point(1256, 12);

this.btnClock.Name = "btnClock";

this.btnClock.Size = new System.Drawing.Size(110, 51);

this.btnClock.TabIndex = 29;

this.btnClock.Text = "Clock";

this.btnClock.UseVisualStyleBackColor = true;

this.btnClock.Click += new System.EventHandler(this.btnClock_Click);

//

// Form1

//

this.AutoScaleDimensions = new System.Drawing.SizeF(16F, 31F);

this.AutoScaleMode = System.Windows.Forms.AutoScaleModeMode.Font;

this.BackColor = System.Drawing.Color.FromArgb(((int)((byte)(91))),
((int)((byte)(107))), ((int)((byte)(131))));

this.ClientSize = new System.Drawing.Size(1554, 826);

this.Controls.Add(this.btnClock);

this.Controls.Add(this.button25);

```

```

        this.Controls.Add(this.btnK3);

        this.Controls.Add(this.button23);

        this.Controls.Add(this.button22);

        this.Controls.Add(this.button21);

        this.Controls.Add(this.btnShowDials);

        this.Controls.Add(this.btnK2);

        this.Controls.Add(this.btnTableau);

        this.Controls.Add(this.btnK1);

        this.Controls.Add(this.btnBorder);

        this.Controls.Add(this.btnStep);

        this.Controls.Add(this.btnStart);

        this.Controls.Add(this.btn48);

        this.Controls.Add(this.btn4);

        this.Controls.Add(this.btn3);

        this.Controls.Add(this.btn12);

        this.Controls.Add(this.btn7);

        this.Controls.Add(this.btnCalendar);

        this.Controls.Add(this.btn44);

        this.Controls.Add(this.btn8);

        this.Controls.Add(this.btn77);

        this.Controls.Add(this.btn6p5);

        this.Controls.Add(this.btn57);

        this.Controls.Add(this.btn38);

        this.Controls.Add(this.btnPopulateCT);

        this.DoubleBuffered = true;

        this.Name = "Form1";

        this.Text = "Form1";

        this.WindowState = System.Windows.Forms.FormWindowState.Maximized;

        this.Load += new System.EventHandler(this.Form1_Load);

        this.ResumeLayout(false);

    }

#endregion

```

```

        private System.Windows.Forms.Timer timer1;

        private System.Windows.Forms.Button btnPopulateCT;

        private System.Windows.Forms.Button btn38;

        private System.Windows.Forms.Button btn57;

        private System.Windows.Forms.Button btn6p5;

        private System.Windows.Forms.Button btn77;

        private System.Windows.Forms.Button btn8;

        private System.Windows.Forms.Button btn44;

        private System.Windows.Forms.Button btnCalendar;

        private System.Windows.Forms.Button btn7;

        private System.Windows.Forms.Button btn12;

        private System.Windows.Forms.Button btn3;

        private System.Windows.Forms.Button btn4;

        private System.Windows.Forms.Button btn48;

        private System.Windows.Forms.Button btnStart;

        private System.Windows.Forms.Button btnStep;

        private System.Windows.Forms.Button btnBorder;

        private System.Windows.Forms.Button btnK1;

        private System.Windows.Forms.Button btnTableau;

        private System.Windows.Forms.Button btnK2;

        private System.Windows.Forms.Button btnShowDials;

        private System.Windows.Forms.Button button21;

        private System.Windows.Forms.Button button22;

        private System.Windows.Forms.Button button23;

        private System.Windows.Forms.Button btnK3;

        private System.Windows.Forms.Button button25;

        private System.Windows.Forms.Button btnClock;

    }
}

```

Source Code for RowNavigation.cs

```

using System;

using System.Collections.Generic;

using System.Linq;

```

```

using System.Text;

using System.Threading.Tasks;

namespace CMechanics
{
    public struct rowPos
    {
        public int nextRow; //The matrix position to navigate to once the trigger value fires
        public bool positive; //Indicates whether traversal is to the right (+ve) or left (-ve)
    }

    public static class RowNavigation
    {
        public static void LoadK3TraversalRows(Dictionary<int, rowPos> rowInstructions)
        {
            rowPos r1 = new rowPos();

            r1.nextRow = 496;

            r1.positive = true;

            rowInstructions.Add(495, r1);

            rowPos r2 = new rowPos();

            r2.nextRow = 529;

            r2.positive = true;

            rowInstructions.Add(526, r2);

            rowPos r3 = new rowPos();

            r3.nextRow = 562;

            r3.positive = true;

            rowInstructions.Add(560, r3);

            rowPos r4 = new rowPos();

            r4.nextRow = 595;

```

```
r4.positive = true;

rowInstructions.Add(592, r4);

rowPos r5 = new rowPos();
r5.nextRow = 628;
r5.positive = true;

rowInstructions.Add(627, r5);

rowPos r6 = new rowPos();
r6.nextRow = 661;
r6.positive = true;

rowInstructions.Add(658, r6);

rowPos r7 = new rowPos();
r7.nextRow = 694;
r7.positive = true;

rowInstructions.Add(693, r7);

rowPos r8 = new rowPos();
r8.nextRow = 727;
r8.positive = true;

rowInstructions.Add(725, r8);

rowPos r9 = new rowPos();
r9.nextRow = 760;
r9.positive = true;

rowInstructions.Add(760, r9);
```

```

        rowPos r10 = new rowPos();

        r10.nextRow = 793;

        r10.positive = true;

        rowInstructions.Add(789, r10);

        rowPos r11 = new rowPos();

        r11.nextRow = 463;

        r11.positive = true;

        rowInstructions.Add(820, r11);

        return;
    }

    public static void LoadCalendarTraversalRows(Dictionary<int,rowPos> rowInstructions)
    {
        for (int i = 0; i < 23; i++)
        {

            rowPos r1 = new rowPos();

            r1.nextRow = 53 + (i * 33);

            r1.positive = true;

            rowInstructions.Add((33 + (i * 33)), r1);

        }

        //Add exception row

        rowPos r2 = new rowPos();

        r2.nextRow = 845;

        r2.positive = true;

        rowInstructions.Add(826, r2);

        //Add last four rows

        for (int j = 0; j < 2; j++)

```

```

    {
        rowPos r3 = new rowPos();

        r3.nextRow = 878 + (j * 33);

        r3.positive = true;

        rowInstructions.Add((858 + (j * 33)), r3);
    }

    rowPos r4 = new rowPos();

    r4.nextRow = 20;

    r4.positive = true;

    rowInstructions.Add(923, r4);

    return;
}

public static void LoadCoordinateTraversalRows(Dictionary<int, rowPos>
rowInstructions)
{
    rowPos r1 = new rowPos();

    //r1.lastRow = 19;

    r1.nextRow = 33;

    r1.positive = true;

    rowInstructions.Add(20, r1);

    rowPos r2 = new rowPos();

    //r2.lastRow = 53;

    r2.nextRow = 0;

    r2.positive = true;

    rowInstructions.Add(39, r2);

    rowPos r3 = new rowPos();

    //r3.lastRow = 19;

    r3.nextRow = 33;

```

```
r3.positive = true;

rowInstructions.Add(58, r3);

rowPos r4 = new rowPos();
//r4.lastRow = 53;
r4.nextRow = 66;
r4.positive = true;

rowInstructions.Add(77, r4);

rowPos r5 = new rowPos();
//r5.lastRow = 86;
r5.nextRow = 449;
r5.positive = false;

rowInstructions.Add(96, r5);

rowPos r6 = new rowPos();
//r5.lastRow = 86;
r6.nextRow = 416;
r6.positive = false;

rowInstructions.Add(103, r6);

rowPos r7 = new rowPos();
//r5.lastRow = 86;
r7.nextRow = 383;
r7.positive = false;

rowInstructions.Add(110, r7);

rowPos r8 = new rowPos();
```

```
//r5.lastRow = 86;

r8.nextRow = 350;

r8.positive = false;


rowInstructions.Add(117, r8);


rowPos r9 = new rowPos();

//r5.lastRow = 86;

r9.nextRow = 317;

r9.positive = false;


rowInstructions.Add(124, r9);


rowPos r10 = new rowPos();

//r5.lastRow = 86;

r10.nextRow = 284;

r10.positive = false;


rowInstructions.Add(131, r10);


rowPos r11 = new rowPos();

//r5.lastRow = 86;

r11.nextRow = 251;

r11.positive = false;


rowInstructions.Add(138, r11);


rowPos r12 = new rowPos();

//r5.lastRow = 86;

r12.nextRow = 218;

r12.positive = false;


rowInstructions.Add(145, r12);
```

```
rowPos r13 = new rowPos();  
//r5.lastRow = 86;  
r13.nextRow = 185;  
r13.positive = false;  
  
rowInstructions.Add(152, r13);  
  
rowPos r14 = new rowPos();  
//r5.lastRow = 86;  
r14.nextRow = 152;  
r14.positive = false;  
  
rowInstructions.Add(159, r14);  
  
rowPos r15 = new rowPos();  
//r5.lastRow = 86;  
r15.nextRow = 119;  
r15.positive = false;  
  
rowInstructions.Add(166, r15);  
  
rowPos r16 = new rowPos();  
//r5.lastRow = 86;  
r16.nextRow = 240;  
r16.positive = false;  
  
rowInstructions.Add(173, r16);  
  
rowPos r17 = new rowPos();  
//r5.lastRow = 86;  
r17.nextRow = 0;  
r17.positive = true;  
  
rowInstructions.Add(192, r17);
```

```

        return;
    }
}
}

```

Source Code for MatrixColourAssignments.cs

```

using System;

using System.Collections.Generic;

using System.Linq;

using System.Text;

using System.Threading.Tasks;

using System.Windows.Forms;

using System.Drawing;

namespace CMechanics
{
    public static class MatrixColourAssignments
    {
        public static Color coordinateTraversal = Color.Yellow;

        public static Color block38 = Color.FromArgb(41, 110, 171);
        public static Color block57 = Color.FromArgb(106, 170, 227);
        public static Color block6p5 = Color.FromArgb(79, 123, 193);
        public static Color block77 = Color.FromArgb(240, 134, 0);
        public static Color block36 = Color.FromArgb(202, 162, 46);
        public static Color block8 = Color.FromArgb(201, 159, 76);
        public static Color block3 = Color.FromArgb(181, 93, 226);
        public static Color block12 = Color.FromArgb(134, 78, 147);
        private static Color block7 = Color.FromArgb(181, 135, 198);
        private static Color block4 = Color.FromArgb(106, 78, 147);
        private static Color block48 = Color.FromArgb(150, 64, 170);
        //private static Color block365 = Color.FromArgb(71, 145, 86);
    }
}

```

```

private static Color block365 = Color.FromArgb(123, 197, 34);

public static void ColourCoordinateBlock38(Dictionary<int, TextBox> matrix)
{
    for (int i = 1; i < 20; i++)
    {
        matrix[i].BackColor = block38;
    }
    for (int i = 34; i < 53; i++)
    {
        matrix[i].BackColor = block38;
    }
    return;
}

public static void ColourCoordinateBlock57(Dictionary<int, TextBox> matrix)
{
    for (int i = 67; i < 86; i++)
    {
        matrix[i].BackColor = Color.FromArgb(59, 100, 165);
    }
    return;
}

public static void ColourCoordinateBlock6P5(Dictionary<int, TextBox> matrix)
{
    for (int i = 100; i < 106; i++)
    {
        matrix[i].BackColor = block6p5;
    }
    for (int i = 107; i < 112; i++)
    {
        matrix[i].BackColor = block6p5;
    }
}

```

```

        return;
    }

    public static void ColourCoordinateBlock77(Dictionary<int, TextBox> matrix)
    {
        for (int j = 0; j < 11; j++)
        {
            int lower = (j * 33) + 112;
            int upper = (j * 33) + 119;
            for (int i = lower; i < upper; i++)
            {
                matrix[i].BackColor = block77;
            }
        }
        return;
    }

    public static void ColourCoordinateBlock36(Dictionary<int, TextBox> matrix)
    {
        for (int j = 0; j < 3; j++)
        {
            int lower = (j * 33) + 133;
            int upper = (j * 33) + 145;
            for (int i = lower; i < upper; i++)
            {
                matrix[i].BackColor = block36;
            }
        }
        return;
    }

    public static void ColourCoordinateBlock8(Dictionary<int, TextBox> matrix)
    {
        for (int i = 232; i < 240; i++)
        {
            matrix[i].BackColor = block8;
        }
    }

```

```

        return;
    }

    public static void ColourCoordinateBlock3(Dictionary<int, TextBox> matrix)
    {
        for (int i = 241; i < 244; i++)
        {
            matrix[i].BackColor = block3;
        }

        return;
    }

    public static void ColourCoordinateBlock12(Dictionary<int, TextBox> matrix)
    {
        for (int i = 265; i < 277; i++)
        {
            matrix[i].BackColor = block12;
        }

        return;
    }

    public static void ColourCoordinateBlock7(Dictionary<int, TextBox> matrix)
    {
        for (int i = 298; i < 305; i++)
        {
            matrix[i].BackColor = block7;
        }

        return;
    }

    public static void ColourCoordinateBlock4(Dictionary<int, TextBox> matrix)
    {
        for (int i = 306; i < 310; i++)
        {
            matrix[i].BackColor = block4;
        }
    }

```

```

    }

    public static void ColourCoordinateBlock48(Dictionary<int, TextBox> matrix)
    {
        for (int j = 0; j < 4; j++)
        {
            int lower = (j * 33) + 331;
            int upper = (j * 33) + 343;
            for (int i = lower; i < upper; i++)
            {
                matrix[i].BackColor = block48;
            }
        }
    }

    public static void ColourCoordinateBlock365(Dictionary<int, TextBox> matrix)
    {
        for (int j = 0; j < 28; j++)
        {
            int lower = (j * 33) + 20;
            int upper = (j * 33) + 33;
            for (int i = lower; i < upper; i++)
            {
                matrix[i].BackColor = block365;
            }
        }

        matrix[759].BackColor = block365;
    }
}

```